



UI-Venus-2 Technical Report

Venus Team, Ant Group

Multimodal GUI agents have emerged as a promising paradigm for digital task automation, yet transitioning from benchmark-oriented models to dependable real-world applications remains challenging due to limited environment coverage, brittle task construction, and unreliable reward verification. In this work, we present **UI-Venus-2**, a general-purpose foundation GUI agent designed to operate across mobile, web, and desktop environments through a unified closed-loop reasoning–action framework. To bridge the gap toward practical deployment, we jointly scale three critical dimensions: (1) Environments, expanding coverage to 170+ multilingual mobile apps and native desktop OS; (2) Tasks, employing a deep-research pipeline for function-grounded instruction generation; and (3) Verification, adopting trace- and sample-level evaluators with visual keypoints and multi-model voting to ensure reliable RL signals for training. Furthermore, we integrate safety-aware mechanisms to ensure controlled execution of consequential actions. By offering a capable, efficient, and open-source foundation, UI-Venus-2 advances the field toward more generalizable, verifiable, and self-reflective agents for real-world applications.

Code: <https://github.com/inclusionAI/UI-Venus>

Model: <https://huggingface.co/collections/inclusionAI/ui-venus>

Project: <https://ui-venus.github.io/UI-Venus-2>

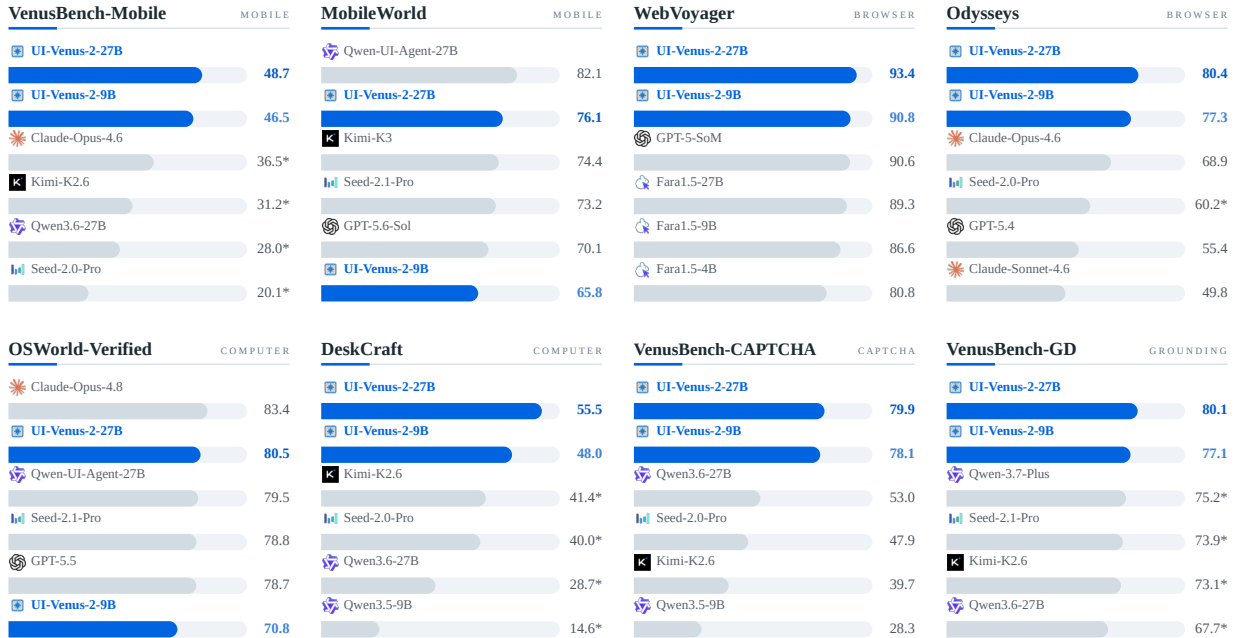


Figure 1 Performances of UI-Venus-2 on GUI-agent benchmarks. Each panel compares UI-Venus-2-27B and UI-Venus-2-9B with some selected strong baselines. We favor standalone end-to-end systems evaluated on the closest available task subset and step budget; source-reported action scaffolds may still differ. MobileWorld uses GUI-only success rate on 117 tasks with 50 steps, WebVoyager uses the refreshed 595-task split, Odysseys uses average rubric score over 200 tasks, VenusBench-CAPTCHA uses micro Pass@1 over all 219 examples, and VenusBench-GD uses English-instruction micro-average accuracy. “*” denotes the results are reproduced by us.

1 Introduction

The ability to autonomously operate digital devices from natural-language instructions has long been a central goal of artificial intelligence. With the rapid development of Multimodal Large Language Models (MLLMs) [Anthropic \(2024\)](#); [Bai et al. \(2025b\)](#); [Zhu et al. \(2025\)](#); [Zhipu-AI \(2025\)](#); [Seed \(2025a\)](#); [Bai et al. \(2025a\)](#); [Ling-Team et al. \(2025a\)](#); [Inclusion-AI et al. \(2025\)](#); [Ling-Team et al. \(2025b\)](#), GUI agents [Gu et al. \(2025\)](#); [Tang et al. \(2025a\)](#); [Ye et al. \(2025\)](#); [Yan et al. \(2025\)](#); [Zhou et al. \(2025b\)](#); [Wang et al. \(2025b\)](#); [Liu et al. \(2024\)](#); [H-Company \(2025\)](#); [Cheng et al. \(2026\)](#); [Zhang et al. \(2026a\)](#) have emerged as a promising interface between high-level user intent and concrete digital execution. Rather than depending on platform-specific APIs or handcrafted workflows, these agents perceive rendered interfaces and interact through human-like actions such as clicking, typing, scrolling, and keyboard operations. This visual, interface-first paradigm makes it possible to automate applications for which structured APIs are unavailable or incomplete, and has consequently attracted substantial attention from both academia and industry.

Despite rapid progress in GUI grounding and navigation, moving from benchmark-oriented agents to dependable real-world applications requires more than improving the accuracy of the next predicted action. Recent user-centric evaluations, such as VenusBench-Mobile [Gong et al. \(2026\)](#) and OS-World [Xie et al. \(2024\)](#), further expose this gap: app-centric and task-homogeneous benchmarks can obscure capability failures, while agents remain brittle under realistic environment variations. First, environment coverage must extend beyond a small collection of applications to multilingual mobile ecosystems, dynamic websites, and full desktop operating systems. Second, scaling interactive environments also requires scalable task construction: generated queries must be grounded in the actual functionality of an application and remain executable under changing interface states. Finally, reinforcement learning is only as reliable as the data quality verifier that supplies its reward. A coarse or brittle verifier may confuse partial progress with true completion, overlook task-critical visual evidence, or expose reward signals that can be exploited by the policy. These challenges make environment coverage, task quality, and data verification tightly coupled components of a practical GUI-agent system. UI-Venus [Gu et al. \(2025\)](#) and UI-Venus-1.5 [Team et al. \(2026c\)](#) established a unified end-to-end framework for GUI grounding and navigation, progressively improving the agent through large-scale GUI mid-training, offline and online reinforcement learning, and the integration of specialized capabilities into a single model. Building on this foundation, we take a further step toward general-purpose computer use by systematically scaling the environments, tasks, and feedback available to the agent.

In this work, We present **UI-Venus-2**, a general-purpose foundation GUI agent designed to operate across mobile applications, web platforms, and desktop operating systems. Rather than treating broader interaction coverage as an isolated scaling problem, UI-Venus-2 follows a unified closed-loop reasoning–action paradigm, in which the agent observes the current interface, reasons about the task state, executes an action, and incorporates subsequent environmental feedback into its next decision.

To develop UI-Venus-2, our key design principle is to scale environment, task distribution and data verification jointly. We substantially expand the executable environment pool to cover diverse mobile, web, and desktop scenarios, while developing task construction pipelines that produce executable and function-grounded tasks at scale. To support reliable reinforcement learning, we further introduce stronger both trace-level and step-level verification that evaluates whether the intended goal has actually been achieved rather than merely detecting superficial progress. Beyond broad environment and task coverage, end-to-end autonomy also requires handling CAPTCHAs in login, registration, and other workflows. This last-mile capability further supports data scaling by preventing verification gates from stalling trajectory collection and blocking downstream states; targeted CAPTCHA data, in turn, strengthens the capability. In parallel, we incorporate safety-aware

mechanisms to ensure that increased operational capability is accompanied by more reliable control over potentially consequential actions.

Moreover, we evaluate UI-Venus-2 across representative mobile, web, and computer-use settings. Figure 1 provides an overview of the evaluation, covering tasks such as navigation, grounding, and CAPTCHA solving, while detailed experimental results and ablations are presented in the following sections. Collectively, UI-Venus-2 advances the UI-Venus family from an action-centric navigation agent toward a broader, verifiable, and self-reflective foundation agent for real-world computer use. Our contributions are summarized as follows:

- **Scaled multilingual mobile-use environments with various and reliable task generation.** We substantially expand the executable mobile environment to cover a broader range of both 100+ Chinese and 70+ English language applications. To generate high-quality tasks at this scale, we introduce a query-generation strategy based on deep-research that grounds task queries in the available application functionality, improving the accuracy, validity, and executability of generated instructions.
- **Computer-use capability built from the ground up.** We extend the UI-Venus family beyond its previous mobile- and web-centered scope by constructing dedicated desktop operating-system capabilities from scratch. Through computer-use data collection and task-specific training, UI-Venus-2 learns to perceive, reason, and act in desktop environments, thereby supporting mobile, web, and OS interaction within a unified end-to-end agent.
- **Trace-level and Sample-level verification.** We introduce a more precise verification mechanism that evaluates task completion using task-relevant visual keypoints rather than relying solely on a coarse holistic judgment of the final interface. We further aggregate judgments from multiple heterogeneous models through voting, improving verification robustness, reducing single-judge bias, and making the reward signal less susceptible to reward hacking.
- **Open-sourced GUI Agent.** We publicly release the full-parameter weight, and evaluation infrastructure of UI-Venus-2, which achieves almost state-of-the-art performance among models of comparable scale across multiple GUI benchmarks. By providing a highly capable yet computationally efficient foundation, we aim to lower the barrier to entry for GUI agent research, facilitate reproducible studies in reinforcement learning and verification, and accelerate community-driven innovation toward more generalizable and reliable computer-use agents.

2 Methodology

2.1 System overview

UI-Venus-2 is a general-purpose foundation GUI agent that combines visual perception and advanced reasoning with reinforcement learning-driven interaction capabilities, enabling autonomous operation across diverse digital environments including mobile applications, desktop operating systems, and web platforms. Given a natural language instruction, the model operates through a unified reasoning–action framework: it observes the rendered interface images, interprets the current visual context, translates high-level user intent into executable GUI actions, and continuously adapts its decisions based on environmental feedback until the task is completed.

To support this interaction paradigm, UI-Venus-2 is trained through a unified pipeline that combines mid-training, offline reinforcement learning, and OPD, with the mid-training and RL components following UI-Venus-1.5 Team et al. (2026c). This training process equips the model with strong GUI understanding, long-horizon planning, precise action grounding, and adaptive error recovery, enabling it to handle complex and dynamic workflows. We leave detailed descriptions of each training stage to the following sections.



Figure 2 System Overview of UI-Venus-2. The figure illustrates the task generation and trajectory collection process of UI-Venus-2. Diverse tasks are constructed to form a multi-domain task pool, and interaction trajectories are collected across mobile, browser, and computer environments, covering a broad range of real-world applications, websites, and desktop software.

Action Spaces: When designing the action space for UI-Venus-2, we additionally design some actions of desktop to better interact with os-use instructions, as shown in the appendix.

Model Init: We initialize our training pipeline from strong open-source multimodal foundation models, Qwen3.5-9B [Qwen Team \(2026a\)](#) and Qwen3.6-27B [Qwen Team \(2026b\)](#), which provide strong priors in visual understanding, multimodal reasoning, and instruction following. Rather than optimizing for CAPTCHA solving as an isolated capability, our framework aims to develop a broader spectrum of visual navigation and computer-use capabilities through a unified mixture of Grounding, CAPTCHA, Mobile, Web, and Computer tasks. These task families are deliberately complementary: Grounding provides fine-grained spatial perception, CAPTCHA offers controlled and verifiable interaction supervision, while Mobile, Web, and Computer tasks constitute the primary sources of navigation-oriented experience in realistic interactive environments. This unified formulation allows knowledge acquired from different domains to be mutually reinforcing rather than learned as independent task-specific skills.

2.2 Stage I: Multimodal Mid-Training

We first perform multimodal mid-training on a large-scale and heterogeneous mixture of synthetic and interaction-oriented data, with Mobile, Web, and OS navigation tasks constituting the dominant component of the training corpus. To obtain diverse and controllable navigation experience, we simulate a broad range of interactive environments spanning different application interfaces, webpages, and operating-system contexts, and collect executable trajectories across these environments. Task queries are constructed through a complementary combination of queries conditioned on curated query seeds, thereby balancing realism, linguistic diversity, and task coverage. The resulting trajectories are further validated through a human–discriminator collaborative verification process, which

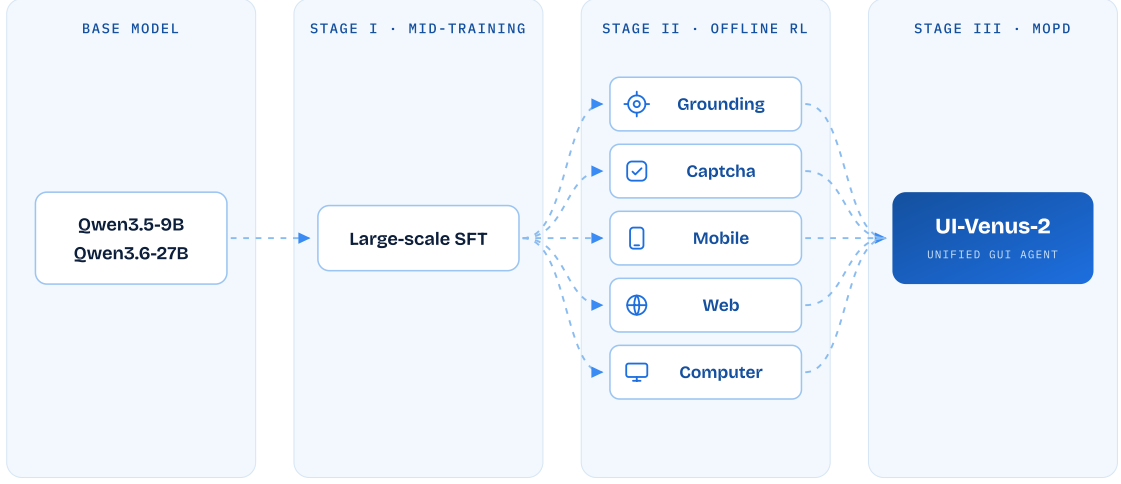


Figure 3 The Three-Stage Pipeline of UI-Venus-2. Following the overall training recipe of UI-Venus-1.5, UI-Venus-2 starts with large-scale trajectory-based mid-training to inject GUI interaction knowledge. The resulting model is then optimized independently for each domain using step-level Offline-RL, covering Grounding, CAPTCHA, Mobile, Web, and Computer tasks. Finally, the domain-specialized models are consolidated into the final UI-Venus-2 model via multi-teacher on-policy distillation.

combines human inspection with automated trajectory-level assessment to filter invalid, ambiguous, or low-quality interactions. Detailed procedures for environment simulation, query construction, trajectory collection, and quality control are provided in the *Data Preparation* section.

2.3 Stage II: Offline Reinforcement Learning

Starting from the mid-trained models, we further perform offline reinforcement learning with task-specific supervision tailored to different interaction scenarios. For Mobile, OS, and Web environments, we construct large-scale step-level RL trajectories, providing supervision at individual interaction steps to optimize state-aware action selection, multi-step navigation, transition consistency, and execution reliability. For CAPTCHA and Grounding, instead we use our programmatic synthesis framework to embed verified CAPTCHA instances and grounding targets into realistic web-page and app background interfaces, allowing the model to learn robust target identification and precise spatial localization under realistic visual clutter. Moreover, the generation process provides reliable action-level correctness and controllable difficulty, enabling dense and scalable RL supervision that would be difficult to obtain from naturally collected interaction data alone. This joint optimization consequently strengthens navigation robustness, spatial accuracy, and action execution consistency across domains.

2.4 Stage III: Multi-teacher On-policy Distillation

Finally, we employ Multi-teacher On-policy Distillation (MOPD) [Xiao et al. \(2026\)](#); [Yan et al. \(2026\)](#) to consolidate capabilities acquired from heterogeneous task distributions and training stages. This stage aims to preserve the broad multimodal reasoning capabilities inherited from the foundation models while integrating specialized skills from Grounding and CAPTCHA with the navigation expertise developed from Mobile, Web, and Computer tasks. The resulting model is not merely a collection of task-specific competencies, but a unified policy where spatial grounding supports element selection, verified interaction strengthens fine-grained execution, and navigation experience enables long-horizon composition.

To mitigate the interference that arises from directly merging independently trained experts, we adopt an on-policy distillation paradigm where teachers score trajectories sampled by the student itself. However, vanilla OPD applies uniform token-level supervision across the entire response, treating reasoning traces and executable actions alike. This is poorly suited to GUI agents because

the action is the sole interface through which the agent interacts with and changes the environment. Although action tokens directly determine execution, they occupy only a small portion of the response and may therefore receive insufficient supervision. Moreover, each action has an explicit structure consisting of an action type and its type-specific parameters. We therefore augment OPD with structured action-aware supervision and teacher-side hints, concentrating the distillation signal on the executable behaviors that govern environment interaction.

Structured Action-Aware Distillation. A GUI response typically contains a reasoning trace followed by a structured action, but only the action is exposed to the environment and determines the resulting state transition. An error in this short action span can invalidate an otherwise coherent response and move the agent away from the intended trajectory. Furthermore, the action exhibits internal dependency: the action type selects the parameter schema, and parameters are meaningful only under the corresponding type. Treating all response tokens uniformly overlooks this asymmetry between reasoning and execution.

We therefore condition the token-level distillation signal on the correctness of the student action while accounting for this structural dependency. Specifically: (1) if the complete action is correct, we suppress its distillation signal as no correction is needed; (2) if the action type is correct but the parameters are not, we strengthen supervision over the action span; (3) if the action type is incorrect, we emphasize the type tokens and mask the downstream parameters, since their meaning depends on the predicted type. For example, an incorrect click location can still be corrected when the action type is `Click`, whereas the parameters of an incorrectly predicted `Scroll` action are semantically irrelevant. This adaptive weighting directs supervision toward the executable component requiring correction rather than allocating it solely according to token frequency.

Teacher-Side Action-Type Conditioning. During training, the routed teacher scores the response sampled by the student. To provide more precise supervision on structured actions, we append a hint $h(z^*)$ regarding the correct action type to the teacher prompt. Crucially, this hint is never included in the student prompt and is unavailable at inference time; it serves only to modulate how the teacher scores student-sampled tokens without altering the student input or inference procedure. The teacher does not generate a separate response but exclusively evaluates the student’s trajectory. The token-level distillation advantage is then computed as:

$$\hat{A}_t^{\text{hint}} = \text{sg}[\log \pi_{T_d}(y_t \mid \mathcal{P}_T(x, z^*), y_{<t}) - \log \pi_\theta(y_t \mid \mathcal{P}_S(x), y_{<t})], \quad (1)$$

where $\hat{A}_t^{\text{hint}}$ is the distillation advantage for the t -th student-sampled token y_t , and $y_{<t}$ denotes its preceding context. T_d is the frozen teacher for domain d , and π_θ is the student policy. The original GUI input is denoted by x , and z^* is the correct action type. $\mathcal{P}_T(x, z^*)$ represents the teacher prompt augmented with the hint, whereas $\mathcal{P}_S(x)$ is the unchanged student prompt. Finally, $\text{sg}[\cdot]$ stops gradients through the advantage computation. By incorporating these GUI-adaptive designs into our multi-teacher setting, MOPD ensures that the distillation signal is effectively concentrated on critical executable behaviors rather than reasoning traces, yielding significantly more robust fusion results across diverse GUI domains.

3 Data Generation and Verification

As shown in Figure 2, mobile, web, and computer-use trajectories share a pipeline for capability discovery, task construction, rollout, and verification. Grounding and CAPTCHA use executable program states to supply target geometry and valid actions directly. Real-environment trajectories retain their task contracts and verification evidence, while synthesized records retain the program state that supplies their labels. We will detail data generation and verification as follows.

3.1 General Generation Pipeline

Our data generation framework operates as a closed-loop iterative system designed to progressively expand task diversity and execution reliability. As illustrated in Figure 2, the pipeline consists of three tightly coupled stages: (1) **Capability Catalog Construction**, which distills application knowledge from heterogeneous sources into a structured, dynamic registry; (2) **Task Construction**, which samples from this catalog to synthesize executable tasks with rigorous validity guarantees; and (3) **Trajectory Collection**, which executes these tasks in real environments and feeds observed outcomes back to update the catalog. This feedback mechanism ensures that task generation is continuously grounded in verified application capabilities and adapts to empirical failure patterns across collection rounds. We detail each stage below.

Capability catalogs. During the cold start phase, Deep Research aggregates application evidence from official documentation, help pages, user discussions, common workflows, and historical tasks. We organize this evidence into an application-specific capability catalog, where each entry records a function’s signature, required objects and preconditions, compatible functions, and current task coverage. Crucially, this catalog is not static; once rollouts begin, it is dynamically updated with observed page states, UI control constraints, entity validations, and failure cases. These runtime observations directly govern the sampling distribution for subsequent rounds, ensuring the catalog reflects the true executable state space rather than merely documented specifications.

Task construction. Guided by the current state of the capability catalog, the generator synthesizes diverse task types, including single-capability, composite, query-based, batch, and scenario-based tasks. Each generated task is formalized as an executable contract that binds the natural language instruction to its application domain, capability tags, initial state, required resources, and expected outcome. To maintain dataset quality, a validity gate enforces strict constraints before execution, rejecting tasks that involve unsupported functions, ambiguous goals, missing dependencies, or unverifiable outcomes. This step ensures that only well-defined and executable tasks enter the trajectory collection stage.

Trajectory collection. The executor interacts with the declared environment through a screenshot-action loop, attempting to fulfill the contracted task. Both successful and failed executions yield structured feedback—including observed functions, invalid entities, unmet preconditions, and specific failure causes—which is written back to the capability catalog. This empirical feedback serves two purposes: it refines the accuracy of capability entries and informs the coverage-aware sampling strategy for the next generation round. Consequently, the pipeline autonomously shifts focus toward under-explored capabilities and previously failing scenarios, enabling systematic improvement over iterations.

3.2 Domain-Specific Process

While the general pipeline (§3.1) provides a unified framework for catalog construction, task generation, and trajectory collection, each application domain presents unique environmental constraints and data requirements. Below, we detail how the core pipeline is instantiated and extended for four distinct domains. Crucially, all domain-specific outputs feed back into the shared capability catalog and adhere to the same validity gating and feedback mechanisms described above; we therefore focus exclusively on domain-adaptive components.

Web Navigation. Web collection requires scalable curation of high-quality interactive environments from the open internet. We construct a website pool by combining public browser-agent benchmarks with the Tranco ranking Pochat et al. (2018), then apply automated accessibility checks followed by Kimi 2.6 scoring on dynamicity, interactivity, content richness, and visual quality. This yields

over 4,000 domains across 19 categories. Additionally, we seed the capability catalog with 45,000 tasks from InSTA-150k-v3 [Trabucco et al. \(2025\)](#), prioritizing entries with rich success criteria annotations. Execution uses real Chrome sessions via a 15-action Playwright interface. Beyond the general validity gate, we apply domain-specific trajectory sanitization: step-level rules remove redundant waits, opposite-direction scrolling, and cyclic action patterns, while recoverable errors are preserved in otherwise successful trajectories to enrich failure-mode coverage in the catalog.

Computer Use. Desktop environments introduce stateful complexity absent in mobile and web domains, requiring deterministic environment setup and long-horizon task management. Each computer-use task is serialized as a TaskSpec containing a controlled desktop snapshot, setup operations, required files/services, source provenance, and an outcome evaluator. A materialization plan provisions the workspace and warmup state; fixture fingerprints deduplicate environments and detect hidden-answer leakage; preflight checks validate initial state integrity before execution enters the general rollout loop. For long-horizon tasks, we introduce hierarchical segmentation: tasks are split into subgoals with independent artifacts and completion checks. A verified exit state from one segment seeds recollection of subsequent segments, with accepted prefixes replayed to preserve context—extending the general trajectory collection mechanism to handle multi-session dependencies.

Synthetic GUI Grounding. Unlike the above domains that collect real-world trajectories, synthetic grounding generates pixel-aligned interaction targets to supplement point-grounding and bounding-box prediction capabilities. We synthesize executable HTML/CSS/JS interfaces from natural-language scenarios, optionally conditioned on personas or reference screenshots [Yang et al. \(2025b\)](#). A single browser pass captures screenshot, DOM, element state, and geometry. Candidate targets are identified via DOM semantics and accessibility attributes, then validated through a nine-point hit test suite covering visibility, viewport clipping, scroll containment, occlusion, and painted-pixel checks; text-node refinement eliminates bounding boxes containing unrelated whitespace. Accepted targets are annotated with coordinates, visible text, ARIA attributes, and local context. Multiple intents may map to one target, while infeasible instructions serve as hard negatives. All records are exported directly for grounding-specific SFT and RL, bypassing the general task-construction stage. Examples are shown in Appendix Figure 4.

Synthetic CAPTCHA. CAPTCHA data requires machine-verifiable ground truth that real-world collection cannot provide. We implement type-specific rule engines covering 70 CAPTCHA types, where each puzzle stores a latent state that deterministically defines its answer, target geometry, valid actions, and solution trace. A separate renderer composites puzzles into mobile panels or webpage contexts, mapping coordinates to the final canvas; puzzle validity is verified before rendering to ensure solvability. The resulting records include rendered observations, normalized geometry, reasoning supervision, and executable action sequences, exported for CAPTCHA-specialized SFT and RL training. This module operates independently of the general catalog but shares the same export format conventions. Examples are shown in Figure 5.

3.3 Trajectory Verification

Reliable reinforcement learning for GUI agents requires verification signals that are both semantically meaningful and temporally granular. Coarse binary success/failure labels derived from final-state checks are insufficient for open-ended tasks, where partial progress, reasoning quality, and task feasibility are equally critical to policy learning. To address this, we introduce a two-level trajectory verification framework that jointly assesses *what* the agent accomplished and *how* it arrived there. Trace-level verification evaluates the overall semantic alignment between the execution trace and the task objective, providing structured quality stratification for data curation. Sample-level verification

complements this by assessing individual action correctness prior to execution, enabling fine-grained step-wise supervision and real-time intervention. Together, these two levels form a unified verification pipeline that supplies robust, multi-resolution reward signals while simultaneously feeding back into upstream task generation to improve data quality at scale.

3.3.1 Trace-level Verification

Rule-based evaluators are effective for verifying file contents, configuration states, or other precisely readable final outcomes, but they struggle with open-ended GUI tasks where success is semantic rather than syntactic. To address this, we introduce **Semantic Guided Verification (SGV)**, a trajectory-level quality inspection strategy that complements rule-based checks by evaluating whether the agent’s execution trace semantically satisfies the task objective based on VLM as a Judge [Sun et al. \(2026\)](#). Crucially, SGV does not rely on the agent’s self-declared success; trajectories that end in failure, timeout, or user escalation are equally eligible for verification.

The SGV pipeline proceeds in five stages: (1) extracting verifiable completion keypoints from the task objective; (2) segmenting the full trajectory into fixed-size windows and overlaying red markers on click (CLK) actions in screenshots to enhance visual grounding; (3) making parallel judgments on which keypoints are satisfied within each window, thereby accumulating process-level evidence; (4) applying hard rules to samples with unambiguous evidence, while routing ambiguous cases to a multimodal final judgment that integrates termination signals, behavioral statistics, per-window explanations, and the final screenshot; and (5) producing a structured report containing the conclusion, reasoning, per-keypoint status, and supporting evidence screenshots.

Each trajectory is classified into one of four categories: *completed* (all critical objectives achieved), *partial* (some objectives met or meaningful progress made toward completion), *infeasible* (task objectively impossible due to external constraints, correctly identified by the agent), and *failed* (no valid progress formed). For example, given the task “Post specified content in a target group,” SGV extracts three keypoints: entering the correct group, filling in the specified title and body, and confirming successful publication. A trajectory that completes the first two keypoints but exhausts its step budget before confirmation would be classified as *partial* rather than *failed*, since it demonstrates substantive progress distinguishable from zero-progress failures.

Importantly, SGV conclusions serve as a quality stratification mechanism for data curation rather than direct training labels or benchmark scores. Completed trajectories enter the high-quality candidate pool; partial trajectories are routed to a review queue for truncation or continuation-based repair; infeasible trajectories provide evidence for refining task feasibility rules; failed trajectories help disentangle deficiencies in task design, environment stability, and agent capability; and newly observed functionalities discovered during verification are fed back into the sub-capability pool to inform future task generation. This closed-loop feedback ensures that verification not only filters data but also actively improves the upstream task construction pipeline.

3.3.2 Sample-level Verification

Complementing trace-level assessment, sample-level verification provides fine-grained supervision at each interaction step. Unlike post-hoc evaluation, we adopt an *a priori* judgment paradigm that assesses action correctness based solely on information available *before* execution—the current screenshot, declared action type and target, agent reasoning, and task goal. This design enables real-time intervention and avoids conflating action quality with downstream page transitions that may be influenced by external factors.

Each step is classified into one of four categories through a two-stage decision process: first checking reasoning-action consistency (whether the agent’s stated intent matches its executed operation), then evaluating task alignment. **(1) Correct:** the action clearly advances the task

and the agent demonstrates confident awareness of its correctness; **(2) Exploratory**: the agent exhibits explicit uncertainty but makes a reasonable attempt toward a plausible candidate path (e.g., probing an unfamiliar interface region), distinguishable from errors by its grounded rationale and from ineffective actions by its induced state change; **(3) Ineffective**: the action yields no substantive contribution, such as clicking non-interactive regions or entering short useless loops; **(4) Incorrect**: the action explicitly deviates from the task objective, continues after completion, or reflects reasoning-action inconsistency (e.g., perceptual hallucination or thought-action mismatch).

Step-level judgments are aggregated into a three-tier trajectory assessment. The first tier determines feasibility: infeasible tasks proceed to identification assessment (correctly reported $\rightarrow$ *infeasible*; unrecognized $\rightarrow$ *failed*). Feasible tasks undergo completion evaluation based on a critical distinction between *generic operations* (app launching, tab switching, scrolling) and *task-specific operations* (entering specific content, selecting particular targets). Only trajectories containing at least one valid task-specific operation qualify as *partial*; those limited to generic operations are classified as *failed*. This fine-grained aggregation ensures that partial credit reflects genuine task-grounded progress, providing a more reliable reward signal for reinforcement learning than coarse binary success/failure labels.

4 Experiments

4.1 Experimental Setup

4.1.1 Implementation details

During inference, we set the sampling temperature to 1.0 and use Qwen3.5’s default visual resolution configuration. For general agentic tasks, reasoning mode (think) is enabled by default, with the complete reasoning history retained and fed back into the context to preserve coherence across multi-step interactions. For GUI grounding tasks, we disable reasoning mode and set the temperature to 0 to enable direct and efficient localization. For CAPTCHA scenarios, our framework further supports multi-action parsing, allowing the model to generate a sequence of operations to complete verification challenges. Additional implementation details for individual benchmarks are provided in their respective experimental sections. For each column in the following tables, the best and second-best results are in **bold** and underlined.

4.1.2 Benchmarks

Mobile Use. We evaluate mobile-use capabilities on six benchmarks: MobileGym [Wu et al. \(2026a\)](#), VenusBench-Mobile [Gong et al. \(2026\)](#), AndroidWorld [Rawles et al. \(2025\)](#), MobileWorld [Kong et al. \(2026\)](#), KnowUBench [Chen et al. \(2026\)](#), and MemGUI [Liu et al. \(2026a\)](#). MobileGym provides a lightweight, browser-based simulation environment with full controllability for everyday mobile use. VenusBench-Mobile shifts the focus toward realistic, user-centric evaluation by assessing general-purpose mobile GUI agents on diverse, dynamic, and fine-grained everyday usage scenarios. AndroidWorld emphasizes realism and reproducibility through dynamically generated, parameterized tasks spanning real-world Android applications. Building on this line of work, MobileWorld introduces a more challenging setting centered on long-horizon, cross-application, user-aware, and hybrid-tool mobile use scenarios within a fully controlled and reproducible environment; in our report, however, we focus specifically on its GUI-based interaction setting. Beyond general mobile-use evaluation, KnowU-Bench targets personalized mobile assistance, testing agents in GUI-based scenarios that require user preference inference, multi-turn clarification, and calibrated proactive support. MemGUI-Bench, in contrast, centers on memory-intensive evaluation, examining whether mobile GUI agents can retain short-term information, accumulate long-term experience, and generalize across sessions and applications.

Computer Use. We evaluate computer-use capabilities on OSWorld-Verified [Xie et al. \(2024\)](#), DeskCraft [Wang et al. \(2026\)](#) and OSWorld 2.0 [Yuan et al. \(2026\)](#). OSWorld-Verified evaluates agents on open-ended tasks in real computer environments, covering interactions with desktop and web applications, operating-system-level operations, and workflows spanning multiple applications. DeskCraft complements OSWorld by targeting long-horizon professional desktop workflows and proactive human-agent collaboration. In addition to complex multi-step tasks, it explicitly evaluates agents’ ability to interact with users during task execution, including seeking clarification under uncertainty, handling user interruptions, and incorporating post-execution feedback. OSWorld 2.0 further extends this evaluation to 108 long-horizon, real-world computer-use workflows spanning everyday and professional scenarios. Its tasks require agents to coordinate hundreds of actions across applications, reason over multiple information sources, adapt to dynamic environments and streaming updates.

Web Navigation. We evaluate web-navigation capabilities on WebVoyager [He et al. \(2024\)](#), Online-Mind2web [Xue et al. \(2025\)](#), REAL [Garg et al. \(2026\)](#) and Odysseys [Jang et al. \(2026\)](#), which provide complementary evaluation settings for real-world web interaction. WebVoyager evaluates end-to-end multimodal web agents on tasks collected from popular real-world websites, emphasizing visual understanding and interaction with live web interfaces. Online-Mind2Web evaluates web agents on realistic tasks across live websites. REAL complements these live-web benchmarks with deterministic, high-fidelity simulations of real websites, enabling controlled and reproducible evaluation of both information-retrieval and state-changing tasks. Odysseys focuses on long-horizon, multi-step workflows that span multiple websites on the live Internet, testing sustained navigation and cross-site reasoning.

GUI Grounding. We evaluate GUI grounding capabilities on VenusBench-GD [Zhou et al. \(2025a\)](#), ScreenSpot-Pro [Li et al. \(2025a\)](#), OSWorld-G-R [Xie et al. \(2025\)](#), and UI-Vision [Nayak et al. \(2025\)](#), which provide complementary evaluation settings for fine-grained visual grounding in real-world interfaces. VenusBench-GD and ScreenSpot-Pro emphasize precise localization of interactive elements across diverse GUI environments, while OSWorld-G-R evaluates grounding in complex desktop tasks with functional interaction requirements. UI-Vision further assesses grounding under visually diverse and challenging interface layouts, providing a comprehensive evaluation of spatial understanding and action-oriented grounding capabilities.

CAPTCHA Solving. We evaluate models on five complementary CAPTCHA benchmarks. We introduce VenusBench-CAPTCHA, a deployment-oriented set of 219 screenshots spanning eight interaction types, from OCR and ordered clicking to visual reasoning and geometric manipulation. We additionally use the complete 1,050-example Spatial-CAPTCHA-Bench [Kharlamova et al. \(2026\)](#), a fixed, category-balanced 1,000-example subset of MCA-Bench [Wu et al. \(2026b\)](#), 319 examples from 15 selected NextGen-CAPTCHAs [Liu et al. \(2026b\)](#) task types, and 16 retained task types from Open CaptchaWorld [Luo et al. \(2025b\)](#). Together, these benchmarks cover spatial reasoning, visual recognition, logical reasoning, target grounding, and executable GUI interaction. Detailed task coverage and selection criteria are provided in Appendix C.

GUI Agent Safety. To assess safety in realistic desktop environments, we evaluate on OS Harm [Kuntz et al. \(2026\)](#) and OSBlind [Ding et al. \(2026\)](#). OS Harm benchmarks safety across three risk categories: deliberate user misuse, third-party prompt injection attacks, and model misbehavior (*e.g.*, accidental costly mistakes). OSBlind further evaluates susceptibility to safety blind spots under benign-looking instructions that lead to unintended harmful outcomes. For both safety benchmarks, we report the Attack Success Rate (ASR), *i.e.*, the fraction of tasks in which the agent carries out the harmful behavior.

Models	MobileGym	VenusBench-Mobile	AndroidWorld	MobileWorld	KnowUBench	MemGUI
<i>General VLMs</i>						
Qwen3.5-9B (Qwen Team, 2026a)	9.0*	15.3*	57.8	18.0(18.0)*	33.3	6.2*
Qwen3.6-27B (Qwen Team, 2026b)	24.6*	28.0*	70.3	36.8(41.9)*	-	25.7*
Claude-Opus-4.6 (Anthropic, 2026a)	-	36.5*	-	44.5	-	-
Kimi-K2.6 (Moonshot AI, 2026a)	38.7*	31.2*	-	55.6	-	39.1
Kimi-K3 (Moonshot AI, 2026b)	-	-	-	74.4	-	-
Seed-2.0-Pro (Seed, 2026)	52.0	20.1*	-	63.2	51.6	65.6*
Seed-2.1-Pro (ByteDance Seed, 2026)	-	-	-	73.2	-	-
GPT-5.6-Sol (OpenAI, 2026)	-	-	-	70.1	-	-
<i>GUI-specific Models</i>						
UI-Venus-1.5-8B (Team et al., 2026c)	18.4*	16.1	73.7	22.2*	26.0	3.9*
UI-Venus-1.5-30B-A3B (Team et al., 2026c)	21.5*	21.5	77.6	17.1	-	10.9*
GUI-Owl-1.5-32B-Instruct (Xu et al., 2026)	20.3*	-	69.8	43.9	-	10.9
MAI-UI-8B (Zhou et al., 2025b)	21.5*	12.7	70.7	27.5	26.0	17.2*
Qwen-UI-Agent-27B (Zhou et al., 2026)	-	-	-	82.1(85.5)	-	-
<i>Ours</i>						
UI-Venus-2-9B	<u>52.7</u>	<u>46.5</u>	<u>80.2</u>	65.8(75.2)	<u>56.5</u>	62.6
UI-Venus-2-27B	60.5	48.7	84.0	<u>76.1(82.9)</u>	59.7	70.3

Table 1 Performance comparison on various mobile GUI benchmarks. VenusBench-Mobile reports success rate on its 149-task primary pool. For MobileWorld, we report GUI-only success rate on 117 tasks under the 50-step setting; values in parentheses, when available, use 100 steps. MemGUI reports Main Results pass@1. “*” denotes the baseline results evaluated or reproduced by us.

4.2 Main Results

In the experiments, we compare UI-Venus-2 models against a broad range of state-of-the-art baselines across two model categories: **(1) General VLMs:** Qwen3.5-9B Qwen Team (2026a), Qwen3.6-27B Qwen Team (2026b), Qwen3.7-Plus Qwen Team (2026c), Claude-Opus-4.6 Anthropic (2026a), Kimi-K2.6 Moonshot AI (2026a), Kimi-K3 Moonshot AI (2026b), Seed-2.0-Pro Seed (2026), Seed-2.1-Pro ByteDance Seed (2026), and GPT-5.6-Sol OpenAI (2026). **(2) GUI-specific Models:** OpenCUA Wang et al. (2025b), GTA1 Yang et al. (2025a), GUI-Owl Ye et al. (2025), UI-TARS-1.5 Seed (2025b), UI-Venus Gu et al. (2025), Holo2 H-Company (2025), Step-GUI Yan et al. (2025), MAI-UI Zhou et al. (2025b), UI-Venus-1.5 Team et al. (2026c), and Qwen-UI-Agent Zhou et al. (2026).

4.2.1 Mobile Use

MobileGym. On MobileGym, our models achieve the strongest performance among all compared methods, with UI-Venus-2-9B reaching 52.7% and UI-Venus-2-27B further improving to 60.5%. The 27B model surpasses the strongest baseline, Seed2.0 Pro (52.0%), by 8.5% points, while substantially outperforming all prior GUI-specific models, whose best result is 21.5%. These results indicate that our models are particularly effective in controllable everyday mobile-use simulations, where robust perception and precise action grounding are critical.

VenusBench-Mobile. On VenusBench-Mobile, which emphasizes realistic, user-centric, and fine-grained evaluation, our models again achieve the best results, reaching 46.5% and 48.7% for the 9B and 27B variants, respectively. The strongest prior baseline is Opus 4.6 (36.5%), followed by Kimi K2.6 (31.2%) and Qwen3.6-27B (28.0%). The substantial margin over both general-purpose and GUI-specific baselines suggests that our approach generalizes better to diverse and dynamic everyday mobile scenarios beyond app-centric benchmark settings.

AndroidWorld. On AndroidWorld, UI-Venus-2-9B and UI-Venus-2-27B achieve 80.2% and 84.0%, respectively, establishing the best results in the table. Notably, several strong baselines already perform well on this benchmark, including UI-Venus-1.5-30B-A3B (77.6%), UI-Venus-1.5-8B (73.7%), and MAI-UI-8B (70.7%), suggesting that AndroidWorld is relatively mature and increasingly close to saturation. Despite this, our 27B model still improves over the previous best by 6.4% points, demonstrating that our approach continues to yield meaningful gains even in a

benchmark where headroom is already limited.

MobileWorld. MobileWorld is substantially more challenging due to its long-horizon and cross-application task structure. Under the GUI-only setting considered in our report, UI-Venus-2-9B achieves 65.8% in the standard 50-step setting and 75.2% in the 100-step setting, while UI-Venus-2-27B reaches 76.1% and 82.9%, respectively. At 50 steps, Qwen-UI-Agent-27B reports 82.1%, while the strongest standalone general-model baselines include Kimi-K3 (74.4%), Seed-2.1-Pro (73.2%), and GPT-5.6-Sol (70.1%). UI-Venus-2-27B therefore remains ahead of these selected general-model baselines, while trailing the strongest specialized agent.

KnowUBench. KnowUBench evaluates personalized mobile assistance, including user preference inference, multi-turn clarification, and calibrated proactive support. On this benchmark, UI-Venus-2-9B and UI-Venus-2-27B achieve 56.5% and 59.7%, respectively, both surpassing the strongest prior baseline, Seed2.0 Pro (51.6%). The improvement is even larger relative to earlier GUI-specific models such as UI-Venus-1.5-8B (26.0%) and MAI-UI-8B (26.0%). These results suggest that our approach improves not only GUI interaction performance but also the user-aware capabilities required for personalized mobile assistance.

MemGUI. On MemGUI, which focuses on short-term retention, long-term experience accumulation, and cross-session learning, UI-Venus-2-27B achieves the best result of 70.3%, while UI-Venus-2-9B reaches 62.6%. The strongest baseline is Seed2.0 Pro (65.6%), whereas other models remain substantially lower, including Kimi K2.6 (39.1%), Qwen3.6-27B (25.7%), and MAI-UI-8B (17.2%). This substantial advantage indicates that our models are particularly strong in memory-intensive mobile GUI scenarios that require retaining and reusing information across actions, applications, and sessions.

4.2.2 Computer Use

Models	OSWorld-Verified	DeskCraft
<i>General VLMs</i>		
Claude-Opus-4.8 (Zhou et al., 2026)	83.4	-
Qwen3.5-9B (Qwen Team, 2026a)	41.8	14.6*
Qwen3.6-27B (Qwen Team, 2026b)	62.0	28.7*
Kimi-K2.6 (Moonshot AI, 2026a)	73.1	41.4*
Seed-2.0-Pro (Seed, 2026)	62.3	40.0*
Seed-2.1-Pro (Zhou et al., 2026)	78.8	-
GPT-5.5 (Zhou et al., 2026)	78.7	-
<i>GUI-specific Models</i>		
GUI-Owl-1.5-32B-Instruct (Xu et al., 2026)	56.5	-
Qwen-UI-Agent-27B (Zhou et al., 2026)	79.5	-
<i>Ours</i>		
UI-Venus-2-9B	70.8	<u>48.0</u>
UI-Venus-2-27B	<u>80.5</u>	55.5

Models	Binary Acc.	Partial Score
<i>General VLMs</i>		
GPT-5.5	13.0	46.7
Claude-Opus-4.7	<u>4.6</u>	<u>20.3</u>
Claude-Sonnet-4.6 (max)	<u>4.6</u>	20.0
Claude-Sonnet-4.6 (medium)	<u>4.6</u>	14.2
MiniMax-M3	1.9	8.2
Kimi-K2.6	1.9	7.1
Qwen3.5-9B	0.0*	2.5*
Qwen3.6-27B	0.0*	3.8*
Seed-2.0-Pro	0.0*	6.3*
<i>Ours</i>		
UI-Venus-2-9B	0.0	7.5
UI-Venus-2-27B	2.8	13.2

Table 2 Performance comparison on computer-use agent benchmarks: **OSWorld-Verified** and **DeskCraft** (left), and **OSWorld 2.0** (right) under the official 150-step budget with 108 tasks. Reported OSWorld-Verified baselines use the 361-task setting in their cited source and may use model-specific action scaffolds. For DeskCraft, we report an author-evaluated aggregate over the 538-task union of the Standard and Interactive splits, which differs from the benchmark’s official split-level reporting. OSWorld 2.0 results report the official Binary Accuracy and Partial Score metrics; baselines are taken from the official leaderboard, possibly with model-specific tool settings, and the reasoning-effort setting is labeled in parentheses for models with multiple official entries. “*” indicates baseline results evaluated by us.

OSWorld-Verified. On OSWorld-Verified, UI-Venus-2-27B achieves 80.5%. This is below the source-reported Claude-Opus-4.8 result of 83.4%, but above Qwen-UI-Agent-27B (79.5%), Seed-2.1-Pro (78.8%), and GPT-5.5 (78.7%). UI-Venus-2-9B obtains 70.8%. Because these reported systems use model-specific action scaffolds—including direct command-line actions for Qwen-UI-Agent—the comparison should be interpreted as a benchmark-level reference rather than a strictly controlled ablation.

Models	WebVoyager	Online-Mind2Web	REAL	Odysseys	
				Avg.	Perfect
General VLMs					
Qwen3.5-9B (Qwen Team, 2026a)	46.9*	27.3*	18.2*	42.6*	13.5*
Qwen3.5-4B (Qwen Team, 2026a)	–	–	–	42.9	10.7
Qwen3.6-27B (Qwen Team, 2026b)	84.3*	55.3*	27.3*	39.5*	18.5*
OpenAI Operator (OpenAI, 2025)	87.0	61.3	–	–	–
GPT-5 (SoM) (Awadallah et al., 2026)	90.6	–	–	–	–
GPT-5.4 (Singh et al., 2025)	–	–	–	55.4	33.5
Seed-2.0-Pro (Seed, 2026)	85.1*	68.5*	74.4*	60.2*	30.1*
GLM-5V-Turbo (Hong et al., 2026)	88.5	–	–	–	–
Claude-Opus-4.6 (Anthropic, 2026a)	88.0	–	–	68.9	44.5
Claude-Sonnet-4.6 (Anthropic, 2026b)	–	–	–	49.8	31.0
Kimi-K2.6 (Moonshot AI, 2026a)	76.8*	–	74.4*	–	–
GUI-specific Models					
UI-TARS-1.5 (Seed, 2025b)	84.8	<u>75.8</u>	–	–	–
UI-Venus-1.5-30B-A3B (Team et al., 2026c)	76.0	–	38.0*	–	–
GUI-Owl-1.5-32B-Thinking (Xu et al., 2026)	82.1	–	44.6*	–	–
MolmoWeb-8B (Gupta et al., 2026)	78.2	35.3	–	–	–
Fara1.5-4B (Awadallah et al., 2026)	80.8	–	–	–	–
Fara1.5-9B (Awadallah et al., 2026)	86.6	63.4	–	–	–
Fara1.5-27B (Awadallah et al., 2026)	89.3	72.3	–	–	–
Ours					
UI-Venus-2-9B	<u>90.8</u>	74.0	<u>76.9</u>	<u>77.3</u>	<u>62.0</u>
UI-Venus-2-27B	93.4	78.3	80.2	80.4	66.3

Table 3 Performance comparison on four live-web benchmarks: **WebVoyager**, **Online-Mind2Web**, **REAL**, and **Odysseys**. The Fara1.5 and GPT-5 (SoM) WebVoyager entries use the refreshed 595-task, 100-step robust protocol and are averaged over three runs; live-site states may vary by evaluation date. For Odysseys, we report both the averaged rubric score (Avg.) and the perfect rubric score (Perfect). "*" indicates our reproduced results.

DeskCraft. On DeskCraft, UI-Venus-2-27B achieves the best overall performance of 55.5%, outperforming the strongest baseline, Kimi-K2.6 (41.4%), by a substantial 14.1 points and Seed-2.0-Pro (40.0%) by 15.5 points. UI-Venus-2-9B ranks second overall with 48.0%, exceeding Kimi-K2.6 by 6.6 points.

OSWorld 2.0. On OSWorld 2.0, we evaluate our models under the official 150-step interaction budget and report both Binary Accuracy and Partial Score. UI-Venus-2-9B and UI-Venus-2-27B achieve Binary Accuracies of 0.0% and 2.8%, and Partial Scores of 7.5% and 13.2%, respectively. The 27B model has a Binary Accuracy of 2.8%, compared with 13.0% for GPT-5.5 and 4.6% for Claude-4.x models. Its Partial Score of 13.2% is higher than Kimi-K2.6 (7.1%), MiniMax-M3 (8.2%), and all evaluated Qwen3.5/3.6 and Seed-2.0-Pro variants, but remains below Claude-Opus-4.7 (20.3%) and Claude-Sonnet-4.6 (20.0%).

4.2.3 Web Navigation

WebVoyager. WebVoyager is a live-web benchmark for evaluating end-to-end web navigation agents. The original benchmark contains 643 tasks across 15 popular real-world websites, covering information seeking, search, and interactive navigation. Following the evaluation protocol of Awadallah et al. (2025), we use a filtered and refreshed version of WebVoyager: approximately 48 tasks that are no longer feasible are removed, while around 50 time-sensitive tasks are updated with valid dates to remain executable in the current web environment. Following the official evaluation protocol, task completion is evaluated using an automatic GPT-4o judge based on the agent’s interaction trajectory and final response. As shown in Table 3, UI-Venus-2-27B achieves the best performance of 93.4%. It outperforms the strongest GUI-based baseline, Fara1.5-27B, by 4.1 points. Compared with GPT-5 (SoM), UI-Venus-2-27B also achieves a 2.8-point improvement. Moreover,

UI-Venus-2-9B reaches 90.8%, retaining strong performance at a substantially smaller model scale.

Online-Mind2Web. Online-Mind2Web is a live-web benchmark designed to evaluate web agents in realistic online environments. It consists of 300 diverse tasks across 136 real-world websites, with task completion assessed by an LLM-based automatic evaluator. UI-Venus-2-27B achieves the best performance of 78.3%. UI-Venus-2-9B model also achieves 74.0%, outperforming Fara1.5-27B despite its substantially smaller scale.

REAL. REAL evaluates web agents on 112 tasks across 11 high-fidelity replicas of real-world web applications. Unlike live-web benchmarks, REAL provides deterministic and reproducible environments while preserving realistic interfaces and state-changing interactions. The tasks span information retrieval, action execution, and their combinations, with action completion verified through programmatic state assertions. UI-Venus-2-27B achieves a new state-of-the-art score of 80.2%, outperforming the strongest baseline of 74.4% by 5.8 points. The 9B model also reaches 76.9%, exceeding the previous best by 2.5 points,

Odysseys. Odysseys is a long-horizon, cross-site benchmark built on the live web, containing 200 tasks derived from realistic human browsing scenarios. Following the official evaluation protocol, we use gemini-3.1-flash-lite-preview as the judge to independently evaluate each rubric item based on the agent’s interaction trajectory. We report two rubric-based metrics: **Averaged**, which measures the average fraction of satisfied rubric items, and **Perfect**, which counts a task as successful only when all rubric items are satisfied. UI-Venus-2-27B achieves the best results on both metrics, reaching 80.4 in averaged rubric score and 66.3 in perfect rubric score, outperforming the strongest baseline by 11.5 and 21.8 points, respectively. The 9B model also attains 77.3 and 62.0, substantially surpassing existing models and demonstrating strong capability.

4.2.4 GUI Grounding

VenusBench-GD. VenusBench-GD is a comprehensive grounding benchmark spanning web, desktop, and mobile UIs, covering both basic localization and reasoning-intensive cases, including *refusal grounding* for infeasible instructions. As shown in Table 4, UI-Venus-2-27B achieves a new state-of-the-art accuracy of 80.1%, improving over UI-Venus-1.5-30B-A3B by 5.1 points. Notably, the 9B model also reaches 77.1%, demonstrating strong grounding performance at a substantially smaller scale.

ScreenSpot-Pro. ScreenSpot-Pro targets high-resolution professional software interfaces, including CAD, development, creative, and office applications, where dense layouts and small interactive elements require precise fine-grained grounding. UI-Venus-2-27B achieves 74.1%, ranking second only to Qwen-UI-Agent-27B (76.6%) and improving by 4.5 points over UI-Venus-1.5-30B-A3B. Notably, it also surpasses strong closed-source flagship foundation models such as Qwen 3.7 Plus (68.9%) and Seed 2.1 Pro (65.3%). The 9B model also attains 73.0%, substantially narrowing the gap to the 27B model.

OSWorld-G-R. OSWorld-G-R evaluates grounding in complex desktop environments, requiring accurate localization of functionally relevant UI elements. UI-Venus-2-27B achieves 79.1%, the highest score among GUI-specific models, while UI-Venus-2-9B reaches 78.5%, demonstrating robust grounding capability across different model scales.

UI-Vision. UI-Vision evaluates grounding under diverse and visually challenging interface layouts. UI-Venus-2-27B achieves 66.9%, second only to Qwen-UI-Agent-27B (70.0%) yet substantially improving over UI-Venus-1.5-30B-A3B by 12.2 points. Together, these results demonstrate the strong and consistent grounding capability of UI-Venus-2 across heterogeneous GUI environments.

Models	Grounding Benchmarks			
	VenusBench-GD	ScreenSpot-Pro	OSworld-G-R	UI-Vision
<i>General VLMs</i>				
Qwen 3.7 Plus (Qwen Team, 2026c)	75.2*	68.9	78.2	<u>68.0</u>
Seed 2.1 Pro (ByteDance Seed, 2026)	73.9*	65.3	78.0	62.0
Kimi-K2.6 (Moonshot AI, 2026a)	73.1*	52.0*	69.7*	51.7*
Qwen3.6-27B (Qwen Team, 2026b)	67.7*	65.2*	76.9*	58.3*
<i>GUI-specific Models</i>				
UI-Venus-Ground-72B (Gu et al., 2025)	70.2	61.9	69.5	36.8
Holo2-30B-A3B (H-Company, 2025)	59.5*	66.1	76.1	40.9*
Step-GUI-4B (Yan et al., 2025)	54.6*	60.0	66.9	30.0*
MAI-UI-8B (Zhou et al., 2025b)	65.2*	65.8	68.6	40.7
MAI-UI-32B (Zhou et al., 2025b)	-	67.9	73.9	47.1
UI-Venus-1.5-30B-A3B (Team et al., 2026c)	75.0	69.6	76.4	54.7
Qwen-UI-Agent-27B (Zhou et al., 2026)	-	76.6	78.5	70.0
<i>Ours</i>				
UI-Venus-2-9B	<u>77.1</u>	73.0	<u>78.5</u>	53.2
UI-Venus-2-27B	80.1	<u>74.1</u>	79.1	66.9

Table 4 Performance comparison on various **Grounding Benchmarks**. VenusBench-GD reports English-instruction micro-average point-in-box accuracy. “*” indicates baselines evaluated or reproduced by us.

4.2.5 CAPTCHA Solving

VenusBench-CAPTCHA. On the deployment-oriented VenusBench-CAPTCHA, models must convert CAPTCHA understanding into complete executable solutions, including ordered clicks, text entry, and geometrically accurate drags. Table 5 shows that UI-Venus-2-27B and UI-Venus-2-9B obtain 79.9% and 78.1% Pass@1, respectively, compared with 53.0% for the strongest general-purpose model, Qwen3.6-27B. The resulting gains of 26.9 and 25.1 percentage points indicate that both variants are effective not only at recognizing CAPTCHA content but also at translating their decisions into valid interface actions. Moreover, the two UI-Venus-2 models differ by only 1.8 points, suggesting that the 9B variant preserves most of the capability required by this end-to-end perception–reasoning–action setting.

MCA-Bench. The category-balanced MCA-Bench subset examines whether performance transfers across a broad task distribution rather than concentrating on a small number of common CAPTCHA formats. Each of its 20 categories contributes 50 examples, giving equal weight to static recognition, target localization, interactive manipulation, and logic-based question answering. On this diverse mixture, UI-Venus-2-27B achieves 79.6% Pass@1 and UI-Venus-2-9B achieves 75.7%, while the strongest general-purpose VLM, Qwen3.6-27B, reaches 51.7%. Thus, even the 9B model improves over the best general-purpose baseline by 24.0 percentage points, with the 27B model extending the margin to 27.9 points. Because no single category dominates the aggregate score, these gains provide evidence that UI-Venus-2 generalizes across substantially different recognition, reasoning, and interaction requirements.

Spatial-CAPTCHA-Bench. Spatial-CAPTCHA-Bench presents a markedly more competitive setting. Over the complete set of 1,050 instances, UI-Venus-2-27B records 48.6% Pass@1, a 4.8-point improvement over Seed-2.0-Pro at 43.8%. UI-Venus-2-9B reaches 42.8%, only 1.0 point below Seed-2.0-Pro but 37.9 points above the similarly sized Qwen3.5-9B. This same-scale comparison reveals a substantial advantage for UI-Venus-2 beyond what can be attributed to parameter count alone. At the same time, scaling UI-Venus-2 from 9B to 27B yields a 5.8-point gain, and even the best result remains below 50%. Together, these observations indicate that reference-frame changes, perspective taking, and multi-step spatial transformations remain important sources of error for all evaluated models, while additional model capacity provides a measurable benefit.

NextGen-CAPTCHAs. The clearest separation from general-purpose VLMs occurs on NextGen-CAPTCHAs. Across 319 examples from 15 retained task types, UI-Venus-2-27B achieves 54.5%

Models	VenusBench-CAPTCHA	MCA-Bench	Spatial-CAPTCHA-Bench	NextGen-CAPTCHAs	Open CaptchaWorld
<i>General VLMs</i>					
Qwen3.5-9B (Qwen Team, 2026a)	28.3	30.4	4.9	2.8	36.4
Qwen3.6-27B (Qwen Team, 2026b)	53.0	51.7	31.0	14.1	47.7
Seed-2.0-Pro (Seed, 2026)	47.9	36.5	43.8	20.4	55.6
Kimi-K2.6 (Moonshot AI, 2026a)	39.7	38.7	24.8	7.2	47.8
Claude-Opus-4.6 (Anthropic, 2026a)	16.0	25.9	9.5	2.8	23.3
<i>Ours</i>					
UI-Venus-2-9B	<u>78.1</u>	<u>75.7</u>	42.8	47.6	50.7
UI-Venus-2-27B	79.9	79.6	48.6	54.5	56.3

Table 5 Performance Comparison across CAPTCHA Benchmarks. All results are Pass@1 percentages, and higher is better. We evaluate on VenusBench-CAPTCHA and four public benchmarks: MCA-Bench Wu et al. (2026b), Spatial-CAPTCHA-Bench Kharlamova et al. (2026), NextGen-CAPTCHAs Liu et al. (2026b), and Open CaptchaWorld Luo et al. (2025b). We use 1,000 sampled MCA-Bench examples, 15 NextGen-CAPTCHAs task types, and 16 Open CaptchaWorld task types; see the appendix for selection details.

Model	OSHarm (ASR ↓)	OSBlind (ASR ↓)
<i>General VLMs</i>		
Qwen3.5-9B (Qwen Team, 2026a)	25.3	79.4
Qwen3.5-27B (Qwen Team, 2026a)	18.0	89.3
Kimi K2.6 (Moonshot AI, 2026a)	32.0	93.6
<i>GUI-specific Models</i>		
EvoCUA-8B (Xue et al., 2026)	39.3	85.3
EvoCUA-32B (Xue et al., 2026)	33.3	90.7
UI-TARS-1.5 (Seed, 2025b)	36.0	83.3
ScaleCUA (Lv et al., 2026)	25.3	84.7
<i>Ours</i>		
UI-Venus-2-9B	11.3	<u>48.8</u>
UI-Venus-2-27B	<u>15.3</u>	47.9

Table 6 Safety evaluation on OSHARM and OS-BLIND benchmarks. ASR denotes Attack Success Rate (%), lower is better).

Pass@1 and UI-Venus-2-9B achieves 47.6%, whereas the strongest general-purpose baseline, Seed-2.0-Pro, reaches only 20.4%. The corresponding improvements of 34.1 and 27.2 percentage points are the largest observed in our evaluation, and the 9B model alone more than doubles the baseline score. These results highlight UI-Venus-2’s strength on CAPTCHAs that combine multiple reasoning primitives, such as counting, rotation, occlusion, viewpoint changes, and path analysis. The 6.9-point gap between the two UI-Venus-2 variants is also the largest model-scale gap among the five benchmarks, suggesting that additional capacity is particularly useful when several visual transformations or intermediate deductions must be integrated into a single prediction.

Open CaptchaWorld. Open CaptchaWorld produces the tightest comparison in Table 5. Across its 16 retained task types, UI-Venus-2-27B reaches 56.3% Pass@1, only 0.7 percentage points above Seed-2.0-Pro at 55.6%. UI-Venus-2-9B ranks third at 50.7%: it trails Seed-2.0-Pro by 4.9 points but remains 2.9 points ahead of the next-best general-purpose model, Kimi-K2.6. Unlike the large margins observed on VenusBench-CAPTCHA, MCA-Bench, and NextGen-CAPTCHAs, this near parity indicates that strong general-purpose VLMs already handle a considerable portion of the recognition, matching, counting, and path-finding skills represented in Open CaptchaWorld. The 27B model’s leading result nevertheless demonstrates broad coverage, while the modest margin and the remaining gap for the 9B variant expose clear headroom on heterogeneous CAPTCHA mechanisms.

4.2.6 GUI Agent Safety

OS-Safety. We evaluate the agent’s safety on two complementary threat settings and report the Attack Success Rate (ASR, lower is better). On OSHarm, which targets explicit threats such as deliberate misuse and prompt injection, UI-Venus-2-9B achieves the lowest ASR of 11.3% and UI-Venus-2-27B follows at 15.3%, both outperforming the strongest baseline, Qwen3.5-27B (18.0%). On OSBlind, where instructions are entirely benign and harm arises from the execution context, all baselines remain highly vulnerable, with ASR ranging from 79.4% to 93.6%. In contrast, UI-Venus-2-9B and UI-Venus-2-27B cut the ASR to 48.8% and 47.9%, reducing the attack success rate by up to nearly half relative to their base counterparts (79.4% and 89.3%, respectively). These results highlight that our models not only resist explicit attacks but also reliably recognize safety blind spots during benign-looking execution, achieving a favorable balance between safety and capability.

5 Related Works

As digital tasks increasingly span mobile devices, web browsers, and desktop operating systems, GUI research is moving from isolated operations toward general-purpose interaction. Practical agents must therefore integrate perception, grounding, reasoning, planning, and interaction capabilities across domains.

Mobile Use. Earlier mobile-use studies primarily relied on static screenshots, recorded trajectories, or fixed benchmarks, which provide reproducible measurements but only partially reflect practical interaction. Recent work increasingly evaluates agents in real applications and dynamic device environments, where interface states, application versions, languages, accounts, and interruptions can affect execution [Qin et al. \(2025\)](#); [Zeng et al. \(2025\)](#); [Sun et al. \(2025b\)](#); [Yang et al. \(2024\)](#); [Sun et al. \(2025a\)](#); [Qiu et al. \(2026a\)](#); [Wang et al. \(2025a\)](#); [Liu et al. \(2024\)](#); [Yan et al. \(2025\)](#); [Cao et al. \(2026\)](#); [Team et al. \(2026b\)](#). This shift places greater demands on state tracking, error recovery, and long-horizon decision making.

Computer Use. Desktop systems and professional software often contain denser interfaces and richer action spaces than mobile applications. Tasks may require keyboard shortcuts, file manipulation, text selection, dragging, window management, and coordination across multiple applications [Li et al. \(2025b\)](#); [Wang et al. \(2025b\)](#); [Yang et al. \(2026\)](#); [Xue et al. \(2026\)](#); [Zhou et al. \(2026\)](#); [Xu et al. \(2026\)](#). Their typically longer action sequences require comprehensive state understanding, planning, memory, progress assessment, recovery, and safe execution.

Web Navigation. The live web is inherently uncertain: network failures may interrupt execution, content can update in real time, and authentication, pop-ups, advertisements, or regional differences can alter the interaction path [Gupta et al. \(2026\)](#); [Awadallah et al. \(2025, 2026\)](#); [Yu et al. \(2025\)](#); [Team et al. \(2026a\)](#); [Ning et al. \(2025\)](#); [Müller and Žunič \(2024\)](#). Cross-site tasks add further uncertainty because different websites evolve independently. Successful navigation therefore requires continuous reassessment and adaptation to changes not observed during training.

GUI Grounding. Grounding becomes more demanding on high-resolution screens and professional software, where precise localization and domain knowledge are needed to distinguish dense or visually similar controls [Lu et al. \(2025\)](#); [Luo et al. \(2025a\)](#); [Liu et al. \(2025\)](#); [Zhou et al. \(2025c\)](#); [Yuan et al. \(2025\)](#); [Tang et al. \(2025b,a\)](#); [Zhang et al. \(2026a\)](#); [Zhou et al. \(2025b\)](#); [Qin et al. \(2025\)](#); [Seed \(2025b\)](#); [Yao et al. \(2026\)](#); [Yan et al. \(2025\)](#); [H-Company \(2025\)](#); [Yang et al. \(2025a\)](#); [Zhang et al. \(2026b\)](#); [Qiu et al. \(2026b\)](#). Targets may also be ambiguous, differ from their descriptions, or be absent from the current screen. Agents must therefore combine visual, functional, and contextual evidence while avoiding incorrect actions when no valid target exists.

CAPTCHA Solving. Modern CAPTCHAs extend beyond distorted-text recognition to ordered clicking, rotation, sliders, dragging, and spatial reasoning. Vision-language models can address

several formats through shared visual understanding and GUI actions [Seed \(2026\)](#); [ByteDance Seed \(2026\)](#); [Qwen Team \(2026d\)](#); [Moonshot AI \(2026a\)](#); [Anthropic \(2025\)](#), although benchmark performance may not reflect deployed interaction patterns. Reliable solving requires fine-grained perception, spatial reasoning, precise grounding, and appropriate action generation.

GUI Agent Safety. Real-world deployment is safety-critical for agents, especially computer-use agents whose actions can have costly consequences for users. Early evaluations primarily examine whether agents can avoid harmful actions in explicitly risky or adversarial settings [Andriushchenko et al. \(2025\)](#); [Zhan et al. \(2024\)](#); [Kuntz et al. \(2026\)](#). Recent work reveals a subtler blind spot: harmful outcomes can also arise from seemingly benign instructions when latent risks are embedded in the environment [Ding et al. \(2026\)](#); [Yin et al. \(2024\)](#). A trustworthy agent should safeguard users regardless of how the underlying risk arises, *e.g.*, from erroneous user instructions, environmental hazards, or agent misalignment.

6 Conclusion

In this work, we presented UI-Venus-2, a GUI agent with state-of-the-art performance achieved through the joint scaling of multilingual environments, desktop computer-use capabilities, and fine-grained verification mechanisms. Beyond benchmark results, we pay particular attention to generalization abilities across diverse platforms such as mobile, web, and desktop operating systems to better serve realistic human everyday use. Looking forward, we hope UI-Venus-2 serves as a reproducible baseline and a collaborative platform for the community to further advance safe, reliable, and truly general-purpose GUI Agents.

7 Contributions

All contributors of UI-Venus-2 are listed in alphabetical order by their last names.

Zhuohan Cai	Jianrong Jiang	Shuheng Shen [†]	Nanjuan Yu
Haoxing Chen	Jianguo Li	Zhongyi Shui	Zhengwen Zeng
Jiaxuan Chen	Runze Li	Weiqiang Wang	Lianrui Zhang
Weizhi Chen	Jinzhen Lin	Ming Wen	Yunzhu Zhang
Changlong Gao	Zhenyu Ma	Zhuoer Xu	Zhe Zhao
Zhangxuan Gu	Changhua Meng [†]	Hang Yan	Beitong Zhou
Yuan Guo	Han Peng	Kaiwen Yang	
Yusong Hu	Xinyu Qiu	Ruilin Yao	

[†]Corresponding Authors: Shuheng Shen(shuheng.ssh@antgroup.com), Changhua Meng(changhua.mch@antgroup.com).

References

- Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, et al. Agentharm: A benchmark for measuring harmfulness of llm agents. In *International Conference on Learning Representations*, volume 2025, pages 79185–79220, 2025.
- Anthropic. Claude computer use. Available at: <https://www.anthropic.com/news/developing-computer-use>, 2024.
- Anthropic. Introducing claude 4, 2025. <https://www.anthropic.com/news/claude-4>.
- Anthropic. Claude opus 4.6 system card, February 2026a. <https://www.anthropic.com/claude-opus-4-6-system-card>.
- Anthropic. Claude sonnet 4.6 system card, February 2026b. <https://www.anthropic.com/claude-sonnet-4-6-system-card>.
- Ahmed Awadallah, Yash Lara, Raghav Magazine, Hussein Mozannar, Akshay Nambi, Yash Pandya, Aravind Rajeswaran, Corby Rosset, Alexey Taymanov, Vibhav Vineet, Spencer Whitehead, and Andrew Zhao. Fara-7b: An efficient agentic model for computer use. *arXiv:2511.19663*, 2025.
- Ahmed Awadallah, Sahil Gupta, Yash Lara, Yadong Lu, Hussein Mozannar, Akshay Nambi, Zach Nussbaum, Yash Pandya, Aravind Rajeswaran, Corby Rosset, et al. Fara-1.5: Scalable learning environments for computer use agents. *arXiv preprint arXiv:2606.20785*, 2026.
- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-v1 technical report. *arXiv preprint arXiv:2511.21631*, 2025a.
- Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-v1 technical report, 2025b. <https://arxiv.org/abs/2502.13923>.
- ByteDance Seed. Seed2.1 model card: Agentic intelligence for productivity, 2026. Technical Report.
- Wanxia Cao, Chengzhen Duan, Pei Fu, Pengzhi Gao, Niu Lian, Fazhan Liu, Hui Liu, Heng Qu, Qinzhuo Wu, Zhehao Yu, et al. Xiaomi-gui-0 technical report. *arXiv preprint arXiv:2606.31410*, 2026.
- Tongbo Chen, Zhengxi Lu, Zhan Xu, Guocheng Shao, Shaohan Zhao, Fei Tang, Yong Du, Kaitao Song, Yizhou Liu, Yuchen Yan, et al. Knowu-bench: Towards interactive, proactive, and personalized mobile agent evaluation. *arXiv preprint arXiv:2604.08455*, 2026.
- Kanzhi Cheng, Zehao Li, Zheng Ma, Nuo Chen, Jialin Cao, Qiushi Sun, Zichen Ding, Fangzhi Xu, Hang Yan, Jiajun Chen, et al. Openmobile: Building open mobile agents with task and trajectory synthesis. *arXiv preprint arXiv:2604.15093*, 2026.
- Xuwei Ding, Skylar Zhai, Linxin Song, Jiate Li, Taiwei Shi, Nicholas Meade, Siva Reddy, Jian Kang, and Jieyu Zhao. The blind spot of agent safety: How benign user instructions expose critical vulnerabilities in computer-use agents, 2026. <https://arxiv.org/abs/2604.10577>.
- Div Garg, Diego Caples, Andis Draguns, Nikil Ravi, Pranav Putta, Naman Garg, Prannay Hebbar, Youngchul Joo, Jindong Gu, Charles London, et al. Real: Benchmarking autonomous agents on deterministic simulations of real websites. *Advances in Neural Information Processing Systems*, 38, 2026.
- Yichen Gong, Zhuohan Cai, Sunhao Dai, Yuqi Zhou, Zhangxuan Gu, Changhua Meng, and Shuheng Shen. Venusbench-mobile: A challenging and user-centric benchmark for mobile gui agents with capability diagnostics. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. <https://arxiv.org/abs/2604.06182>. Oral Presentation.
- Zhangxuan Gu, Zhengwen Zeng, Zhenyu Xu, Xingran Zhou, Shuheng Shen, Yunfei Liu, Beitong Zhou, Changhua Meng, Tianyu Xia, Weizhi Chen, et al. Ui-venus technical report: Building high-performance ui agents with rft. *arXiv preprint arXiv:2508.10833*, 2025.
- Tanmay Gupta, Piper Wolters, Zixian Ma, Peter Sushko, Rock Yuren Pang, Diego Llanes, Yue Yang, Taira Anderson, Boyuan Zheng, Zhongzheng Ren, et al. Molmoweb: Open visual web agent and open data for the open web. *arXiv preprint arXiv:2604.08516*, 2026.
- H-Company. Holo2 - open foundation models for navigation and computer use agents, 2025.
- Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. *arXiv preprint arXiv:2401.13919*, 2024.

- Wenyi Hong, Xiaotao Gu, Ziyang Pan, Zhen Yang, Yuting Wang, Yue Wang, Yuanchang Yue, Yu Wang, Yanling Wang, Yan Wang, et al. Glim-5v-turbo: Toward a native foundation model for multimodal agents. *arXiv preprint arXiv:2604.26752*, 2026.
- Inclusion-AI, Bowen Ma, Cheng Zou, Canxiang Yan, Chunxiang Jin, Chunjie Shen, Chenyu Lian, Dandan Zheng, Fudong Wang, Furong Xu, et al. Ming-flash-omni: A sparse, unified architecture for multimodal perception and generation. *arXiv preprint arXiv:2510.24821*, 2025.
- Lawrence Keunho Jang, Jing Yu Koh, Daniel Fried, and Ruslan Salakhutdinov. Odysseys: Benchmarking web agents on realistic long horizon tasks. *arXiv preprint arXiv:2604.24964*, 2026.
- Arina Kharlamova, Bowei He, Chen Ma, and Xue Liu. Spatial CAPTCHA: Generatively benchmarking spatial reasoning for human-machine differentiation. In *The Fourteenth International Conference on Learning Representations*, 2026. <https://openreview.net/forum?id=8LAc5kzjhz>.
- Quyu Kong, Xu Zhang, Zhenyu Yang, Nolan Gao, Chen Liu, Panrong Tong, Chenglin Cai, Hanzhang Zhou, Jianan Zhang, Liangyu Chen, et al. Mobileworld: Benchmarking autonomous mobile agents in agent-user interactive and mcp-augmented environments. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6142–6167, 2026.
- Thomas Kuntz, Agatha Duzan, Hao Zhao, Francesco Croce, Zico Kolter, Nicolas Flammarion, and Maksym Andriushchenko. Os-harm: A benchmark for measuring safety of computer use agents. *Advances in Neural Information Processing Systems*, 38, 2026.
- Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use, 2025a.
- Pengxiang Li, Zechen Hu, Zirui Shang, Jingrong Wu, Yang Liu, Hui Liu, Zhi Gao, Chenrui Shi, Bofei Zhang, Zihao Zhang, Xiaochuan Shi, Zedong Yu, Yuwei Wu, Xinxiao Wu, Yunde Jia, Liuyu Xiang, Zhaofeng He, and Qing Li. Efficient multi-turn rl for gui agents via decoupled training and adaptive data curation. *arXiv preprint arXiv:2509.23866*, 2025b. <https://arxiv.org/abs/2509.23866>.
- Ling-Team, Ang Li, Ben Liu, Binbin Hu, Bing Li, Bingwei Zeng, Borui Ye, Caizhi Tang, Changxin Tian, Chao Huang, et al. Every activation boosted: Scaling general reasoner to 1 trillion open language foundation. *arXiv preprint arXiv:2510.22115*, 2025a.
- Ling-Team, Anqi Shen, Baihui Li, Bin Hu, Bin Jing, Cai Chen, Chao Huang, Chao Zhang, Chaokun Yang, Cheng Lin, et al. Every step evolves: Scaling reinforcement learning for trillion-scale thinking model. *arXiv preprint arXiv:2510.18855*, 2025b.
- Guangyi Liu, Pengxiang Zhao, Yaozhen Liang, Qinyi Luo, Shunye Tang, Yuxiang Chai, Weifeng Lin, Han Xiao, WenHao Wang, Siheng Chen, et al. Memgui-bench: Benchmarking memory of mobile gui agents in dynamic environments. *arXiv preprint arXiv:2602.06075*, 2026a.
- Jiacheng Liu, Yaxin Luo, Jiacheng Cui, Xinyi Shang, Xiaohan Zhao, and Zhiqiang Shen. Next-gen CAPTCHAs: Leveraging the cognitive gap for scalable and diverse GUI-agent defense. In *Forty-Third International Conference on Machine Learning*, July 2026b. <https://openreview.net/forum?id=wPKGirrtz>.
- Xiao Liu, Bo Qin, Dongzhu Liang, Guang Dong, Hanyu Lai, Hanchen Zhang, Hanlin Zhao, Iat Long Iong, Jiadai Sun, Jiaqi Wang, et al. Autoglm: Autonomous foundation agents for guis. *arXiv preprint arXiv:2411.00820*, 2024.
- Yuhang Liu, Pengxiang Li, Congkai Xie, Xavier Hu, Xiaotian Han, Shengyu Zhang, Hongxia Yang, and Fei Wu. Infigui-rl: Advancing multimodal gui agents from reactive actors to deliberative reasoners. 2025. <https://arxiv.org/abs/2504.14239>.
- Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Guanqing Xiong, and Hongsheng Li. Ui-rl: Enhancing efficient action prediction of gui agents by reinforcement learning. 2025. <https://arxiv.org/abs/2503.21620>.
- Run Luo, Lu Wang, Wanwei He, and Xiaobo Xia. Gui-rl : A generalist rl-style vision-language action model for gui agents. 2025a. <https://arxiv.org/abs/2504.10458>.
- Yaxin Luo, Zhaoyi Li, Jiacheng Liu, Jiacheng Cui, Xiaohan Zhao, and Zhiqiang Shen. Open CaptchaWorld: A comprehensive web-based platform for testing and benchmarking multimodal LLM agents. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025b. doi: 10.52202/085713-3006. <https://doi.org/10.52202/085713-3006>. Datasets and Benchmarks Track.
- Bowen Lv, Xiao Liu, Yanyu Ren, Hanyu Lai, Bohao Jing, Hanchen Zhang, Yanxiao Zhao, Shuntian Yao, Jie Tang, and Yuxiao Dong. Scalecua: Scaling computer use agents with verifiable task synthesis and efficient online rl, 2026. <https://arxiv.org/abs/2607.11185>.
- Moonshot AI. Kimi k2.6. Hugging Face Model Card, 2026a. moonshotai/Kimi-K2.6.
- Moonshot AI. Kimi k3, July 2026b. <https://www.kimi.com/blog/kimi-k3>.
- Magnus Müller and Gregor Žunič. Browser use: Enable ai to control your browser, 2024. <https://github.com/browser-use/browser-use>.

- Shravan Nayak, Xiangru Jian, Kevin Qinghong Lin, Juan A Rodriguez, Montek Kalsi, Rabiul Awal, Nicolas Chapados, M Tamer Özsu, Aishwarya Agrawal, David Vazquez, et al. Ui-vision: A desktop-centric gui benchmark for visual perception and interaction. *arXiv preprint arXiv:2503.15661*, 2025.
- Liangbo Ning, Ziran Liang, Zhuohang Jiang, Haohao Qu, Yujuan Ding, Wenqi Fan, Xiao yong Wei, Shanru Lin, Hui Liu, Philip S. Yu, and Qing Li. A survey of webagents: Towards next-generation ai agents for web automation with large foundation models, 2025. <https://arxiv.org/abs/2503.23350>.
- OpenAI. Introducing operator. Available at: <https://openai.com/index/introducing-operator/>, 2025.
- OpenAI. Introducing gpt-5.6, July 2026. <https://openai.com/index/gpt-5-6/>.
- Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Wouter Joosen, et al. Tranco: A research-oriented top sites ranking hardened against manipulation. *arXiv preprint arXiv:1806.01156*, 2018.
- Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjuan Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. Ui-tars: Pioneering automated gui interaction with native agents, 2025. <https://arxiv.org/abs/2501.12326>.
- Xinyu Qiu, Heng Jia, Zhengwen Zeng, Shuheng Shen, Changhua Meng, Yi Yang, and Linchao Zhu. Unified generation and self-verification for vision-language models via advantage decoupled preference optimization. *arXiv preprint arXiv:2601.01483*, 2026a.
- Xinyu Qiu, Yunzhu Zhang, Heng Jia, Shuheng Shen, Changhua Meng, and Linchao Zhu. Vista: View-consistent self-verified training for gui grounding, 2026b. <https://arxiv.org/abs/2606.14579>.
- Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026a. <https://qwen.ai/blog?id=qwen3.5>.
- Qwen Team. Qwen3.6-27B: Flagship-level coding in a 27B dense model, April 2026b. <https://qwen.ai/blog?id=qwen3.6-27b>.
- Qwen Team. Qwen3.7-Plus: Multimodal agent intelligence, May 2026c. <https://qwen.ai/blog?id=qwen3.7-plus>.
- Qwen Team. Qwen3.8-max: A new bar for coding and cowork, August 2026d. <https://qwen.ai/blog?id=qwen3.8>.
- Chris Rawles, Sarah Clincemallie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyi Campbell-Ajala, et al. Androidworld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations*, volume 2025, pages 406–441, 2025.
- Bytedance Seed. Seed1. 8 model card: Towards generalized real-world agency, 2025a. <https://lf3-static.bytednsdoc.com/obj/eden-cn/lapzild-tss/ljhwZthlaukjlkulzlp/research/Seed-1.8-Modelcard.pdf>.
- ByteDance Seed. Ui-tars-1.5. <https://seed-tars.com/1.5>, 2025b.
- Bytedance Seed. Seed2. 0 model card: Towards intelligence frontier for real-world complexity. *arXiv preprint arXiv:2607.00248*, 2026.
- Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- Qiushi Sun, Kanzhi Cheng, Zichen Ding, Chuanyang Jin, Yian Wang, Fangzhi Xu, Zhenyu Wu, Chengyou Jia, Liheng Chen, Zhoumianze Liu, Ben Kao, Guohao Li, Junxian He, Yu Qiao, and Zhiyong Wu. Os-genesis: Automating gui agent trajectory construction via reverse task synthesis, 2025a. <https://arxiv.org/abs/2412.19723>.
- Qiushi Sun, Kanzhi Cheng, Yian Wang, Bowen Yang, Hang Yan, Liheng Chen, Fangzhi Xu, Zichen Ding, Nuo Chen, Jialin Cao, et al. Osreward: Instituting standardized evaluation for cross-platform computer-use reward models. *arXiv preprint arXiv:2607.28609*, 2026.
- Yuchen Sun, Shanhui Zhao, Tao Yu, Hao Wen, Samith Va, Mengwei Xu, Yuanchun Li, and Chongyang Zhang. Gui-xplore: Empowering generalizable gui agents with one exploration, 2025b. <https://arxiv.org/abs/2503.17709>.
- Fei Tang, Zhangxuan Gu, Zhengxi Lu, Xuyang Liu, Shuheng Shen, Changhua Meng, Wen Wang, Wenqi Zhang, Yongliang Shen, Weiming Lu, Jun Xiao, and Yueting Zhuang. Gui-g²: Gaussian reward modeling for gui grounding, 2025a. <https://arxiv.org/abs/2507.15846>.
- Jiaqi Tang, Yu Xia, Yi-Feng Wu, Yuwei Hu, Yuhui Chen, Qing-Guo Chen, Xiaogang Xu, Xiangyu Wu, Hao Lu, Yanqing Ma, Shiyin Lu, and Qifeng Chen. Lpo: Towards accurate gui agent interaction via location preference optimization, 2025b. <https://arxiv.org/abs/2506.09373>.
- AIMAE Team, Tianxiang Chen, Yan Cheng, Zhangye Han, Xiaowei Li, Chang Liu, Cheng Liu, Zhongqiang Ma, Long Peng, Xiaobing Tu, Yinggui Wang, Hongliang Wei, Chen Wu, Daiping Xin, Kunyu Zhou, Pengyang Zhou, Peiyuan Chen, Ziyuan Chen, Yutao Deng, Chunyu Dong, Xiangyu Fu, Yicheng Feng, Ruian He, Haochen Li, Miancan Liu, Zhengqin Liu, Wei Peng, Jinkui Ren, Haoyu Tan, Dong Xiao, Rongkun Xue, Shujian Yang, Xianhang Ye, Ziqi Yuan, Ziyang Yu, Linghan Zhang, Xiantao Zhang,

- Xuanpu Zhao, Yinan Zhao, Zhenghui Zhao, Bin Zhu, and Likai Zou. Wuying-browser-agent: Real-world centric fundamental long-horizon browser agents, 2026a. <https://arxiv.org/abs/2608.17319>.
- Hy Vision Team, Huawei Shen, Zhengyang Tang, Shangpin Peng, Liang Wu, Anran Zhang, Weinong Wang, Yiduo Guo, Chenxin Li, Zhengyao Fang, et al. Hymobileagent: Data-environment co-scaling for efficient gui agents. *arXiv preprint arXiv:2607.14548*, 2026b.
- Venus Team, Changlong Gao, Zhangxuan Gu, Yulin Liu, Xinyu Qiu, Shuheng Shen, Yue Wen, Tianyu Xia, Zhenyu Xu, Zhengwen Zeng, et al. Ui-venus-1.5 technical report. *arXiv preprint arXiv:2602.09082*, 2026c.
- Brandon Trabucco, Gunnar Sigurdsson, Robinson Piramuthu, and Ruslan Salakhutdinov. Insta: Towards internet-scale training for agents, 2025.
- Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Junting Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, et al. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025a.
- Wenkai Wang, Tao Xiong, Jingchen Ni, Yunpeng Bao, Xiyun Li, Tianqi Liu, Hongcan Guo, Zilong Huang, and Shengyu Zhang. Deskcrafter: Benchmarking desktop agents on professional workflows and human-in-the-loop collaboration. *arXiv preprint arXiv:2606.03103*, 2026.
- Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, Zhennan Shen, Zhuokai Li, Ryan Li, Xiaochuan Li, Junda Chen, Boyuan Zheng, Peihang Li, Fangyu Lei, Ruisheng Cao, Yeqiao Fu, Dongchan Shin, Martin Shin, Jiarui Hu, Yuyan Wang, Jixuan Chen, Yuxiao Ye, Danyang Zhang, Dikang Du, Hao Hu, Huarong Chen, Zaida Zhou, Yipu Wang, Heng Wang, Diyi Yang, Victor Zhong, Flood Sung, Y. Charles, Zhilin Yang, and Tao Yu. Opencua: Open foundations for computer-use agents, 2025b. <https://arxiv.org/abs/2508.09123>.
- Dingbang Wu, Rui Hao, Haiyang Wang, Shuzhe Wu, Han Xiao, Zhenghong Li, Bojiang Zhou, Zheng Ju, Zichen Liu, Lue Fan, et al. Mobilegym: A verifiable and highly parallel simulation platform for mobile gui agent research. *arXiv preprint arXiv:2605.26114*, 2026a.
- Zonglin Wu, Yule Xue, Yaoyao Feng, Xiaolong Wang, and Yiren Song. MCA-Bench: A multimodal benchmark for evaluating CAPTCHA robustness against VLM-based attacks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(44): 38039–38047, March 2026b. doi: 10.1609/aaai.v40i44.41141. <https://doi.org/10.1609/aaai.v40i44.41141>.
- Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, et al. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*, 2026.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.
- Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, Yiheng Xu, Junli Wang, Doyen Sahoo, Tao Yu, and Caiming Xiong. Scaling computer-use grounding via user interface decomposition and synthesis, 2025. <https://arxiv.org/abs/2505.13227>.
- Haiyang Xu, Xi Zhang, Haowei Liu, Junyang Wang, Zhaozai Zhu, Shengjie Zhou, Xuhao Hu, Feiyu Gao, Junjie Cao, Zihua Wang, et al. Mobile-agent-v3. 5: Multi-platform fundamental gui agents. *arXiv preprint arXiv:2602.16855*, 2026.
- Taofeng Xue, Chong Peng, Mianqiu Huang, Linsen Guo, Tiancheng Han, Haozhe Wang, Jianing Wang, Xiaocheng Zhang, Xin Yang, Dengchang Zhao, et al. Evocua: Evolving computer use agents via learning from scalable synthetic experience. *arXiv preprint arXiv:2601.15876*, 2026.
- Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. An illusion of progress? assessing the current state of web agents. In *Second Conference on Language Modeling*, 2025. <https://openreview.net/forum?id=6jZi4HSs6o>.
- Hang Yan, Zhangxuan Gu, Beitong Zhou, Jiaxuan Chen, Runze Li, Yusong Hu, Shuheng Shen, and Changhua Meng. Maga: Multi-platform self-fusion of gui agents via structured action distillation. *arXiv preprint arXiv:2607.29320*, 2026.
- Haolong Yan, Jia Wang, Xin Huang, Yeqing Shen, Ziyang Meng, Zhimin Fan, Kaijun Tan, Jin Gao, Lieyu Shi, Mi Yang, et al. Step-gui technical report. *arXiv preprint arXiv:2512.15431*, 2025.
- Yan Yang, Dongxu Li, Yutong Dai, Yuhao Yang, Ziyang Luo, Zirui Zhao, Zhiyuan Hu, Junzhe Huang, Amrita Saha, Zeyuan Chen, Ran Xu, Liyuan Pan, Caiming Xiong, and Junnan Li. Gta1: Gui test-time scaling agent, 2025a. <https://arxiv.org/abs/2507.05791>.
- Yue Yang, Ajay Patel, Matt Deitke, Tanmay Gupta, Luca Weihs, Andrew Head, Mark Yatskar, Chris Callison-Burch, Ranjay Krishna, Aniruddha Kembhavi, et al. Scaling text-rich image understanding via code-guided synthetic multimodal data generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17486–17505, 2025b.
- Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. Aria-ui: Visual grounding for gui instructions, 2024. <https://arxiv.org/abs/2412.16256>.

- Yuhao Yang, Zhen Yang, Zi-Yi Dou, Anh Nguyen, Keen You, Omar Attia, Andrew Szot, Michael Feng, Ram Ramrakhya, Alexander Toshev, Chao Huang, Yinfei Yang, and Zhe Gan. Ultracua: A foundation model for computer use agents with hybrid action, 2026. <https://arxiv.org/abs/2510.17790>.
- Ruilin Yao, Shegnwu Xiong, Tianyu Zou, Shili Xiong, and Yi Rong. Autofocus: Uncertainty-aware active visual search for gui grounding. *arXiv preprint arXiv:2605.02630*, 2026.
- Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al. Mobile-agent-v3: Fundamental agents for gui automation. *arXiv preprint arXiv:2508.15144*, 2025.
- Sheng Yin, Xianghe Pang, Yuanzhuo Ding, Menglan Chen, Yutong Bi, Yichen Xiong, Wenhao Huang, Zhen Xiang, Jing Shao, and Siheng Chen. Safeagentbench: A benchmark for safe task planning of embodied llm agents. *arXiv preprint arXiv:2412.13178*, 2024.
- Tao Yu, Zhengbo Zhang, Zhiheng Lyu, Junhao Gong, Hongzhu Yi, Xinming Wang, Yuxuan Zhou, Jiabing Yang, Ping Nie, Yan Huang, and Wenhui Chen. Browseragent: Building web agents with human-inspired web browsing actions, 2025. <https://arxiv.org/abs/2510.10666>.
- Mengqi Yuan, Zilong Zhou, Xinzhuang Xiong, Weiming Wu, Jiayang Sun, Jiamin Song, Kaiqian Cui, Bowen Wang, Haoyuan Wu, Yitong Li, Dunjie Lu, Haikong Lu, Qi Zhen, Xinyuan Wang, Jiaqi Deng, Yuhao Yang, Cheng Chen, Boyuan Zheng, Alex Su, Xiao Yu, Hao Zou, Saaket Agashe, Xing Han Lu, Manpreet Kaur, Zhengyang Qi, Vincent Sunn Chen, Frederic Sala, Dayiheng Liu, Junyang Lin, Zhou Yu, Yu Su, Siva Reddy, Xin Eric Wang, Peng Qi, Tianbao Xie, and Tao Yu. Oworld 2.0: Benchmarking computer use agents on long-horizon real-world tasks. *arXiv preprint arXiv:2606.29537*, 2026.
- Xinbin Yuan, Jian Zhang, Kaixin Li, Zhuoxuan Cai, Lujian Yao, Jie Chen, Enguang Wang, Qibin Hou, Jinwei Chen, Peng-Tao Jiang, and Bo Li. Enhancing visual grounding for gui agents via self-evolutionary reinforcement learning. 2025. <https://arxiv.org/abs/2505.12370>.
- Zhixiong Zeng, Jing Huang, Liming Zheng, Wenkang Han, Yufeng Zhong, Lei Chen, Longrong Yang, Yingjie Chu, Yuzhi He, and Lin Ma. Uitrone: Foundational gui agent with advanced perception and planning. *arXiv preprint arXiv:2508.21767*, 2025.
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, 2024.
- Le Zhang, Yixiong Xiao, Xinjiang Lu, Jingjia Cao, Yusai Zhao, Jingbo Zhou, Lang An, Zikan Feng, Wanxiang Sha, Yu Shi, Congxi Xiao, Jian Xiong, Yankai Zhang, Hua Wu, and Haifeng Wang. Omegause: Building a general-purpose gui agent for autonomous task execution, 2026a. <https://arxiv.org/abs/2601.20380>.
- Yunzhu Zhang, Zeyu Pan, Zhengwen Zeng, Shuheng Shen, Changhua Meng, and Linchao Zhu. Mvp: Multiple view prediction improves gui grounding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026b.
- Zhipu-AI. Glm-4.5v. Available at: <https://docs.z.ai/guides/vlm/glm-4.5v>, 2025.
- Beitong Zhou, Zhexiong Huang, Yuan Guo, Zhangxuan Gu, Tianyu Xia, Zichen Luo, Fei Tang, Dehan Kong, Yanyi Shang, Suling Ou, et al. Venusbench-gd: A comprehensive multi-platform gui benchmark for diverse grounding tasks. *arXiv preprint arXiv:2512.16501*, 2025a.
- Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, et al. Mai-ui technical report: Real-world centric foundation gui agents. *arXiv preprint arXiv:2512.22047*, 2025b.
- Hanzhang Zhou, Panrong Tong, Xu Zhang, Quyu Kong, Chenglin Cai, Tianyu Xia, Gongjie Zhang, Jianan Zhang, Long Li, Long Chen, et al. Qwen-ui-agent technical report: Toward next-generation real-world centric foundation gui agents. *arXiv preprint arXiv:2607.28227*, 2026.
- Yuqi Zhou, Sunhao Dai, Shuai Wang, Kaiwen Zhou, Qinglin Jia, and Jun Xu. Gui-g1: Understanding r1-zero-like training for visual grounding in gui agents. 2025c. <https://arxiv.org/abs/2505.15810>.
- Jinguo Zhu, Weiyun Wang, Zhe Chen, Zhaoyang Liu, Shenglong Ye, Lixin Gu, Hao Tian, Yuchen Duan, Weijie Su, Jie Shao, Zhangwei Gao, Erfei Cui, Xuehui Wang, Yue Cao, Yangzhou Liu, Xingguang Wei, Hongjie Zhang, Haomin Wang, Weiye Xu, Hao Li, Jiahao Wang, Nianchen Deng, Songze Li, Yinan He, Tan Jiang, Jiapeng Luo, Yi Wang, Conghui He, Botian Shi, Xingcheng Zhang, Wenqi Shao, Junjun He, Yingdong Xiong, Wenwen Qu, Peng Sun, Penglong Jiao, Han Lv, Lijun Wu, Kaipeng Zhang, Huipeng Deng, Jiaye Ge, Kai Chen, Limin Wang, Min Dou, Lewei Lu, Xizhou Zhu, Tong Lu, Dahua Lin, Yu Qiao, Jifeng Dai, and Wenhui Wang. Internvl3: Exploring advanced training and test-time recipes for open-source multimodal models, 2025. <https://arxiv.org/abs/2504.10479>.

A Action Space and Prompt Templates

A.1 Action Space

Action	Definition
Shared Actions (All Platforms)	
Click(point=(x, y))	Click at coordinates (x, y).
Drag(start=(x1, y1), end=(x2, y2))	Drag from (x1, y1) to (x2, y2).
Swipe(start=(x1, y1), end=(x2, y2))	Scroll by swiping from (x1, y1) to (x2, y2).
DoubleClick(point=(x, y))	Perform a double-click/tap at coordinates (x, y).
LongPress(point=(x, y))	Long press at coordinates (x, y) to trigger extra options.
Type(content=)	Type the specified content.
Wait()	Wait for loading.
CallUser(content=)	Request user takeover or additional information.
Finished(content=)	Mark the task as completed, with optional information.
Mobile	
PressBack()	Press the ‘back’ button.
PressHome()	Press the ‘home’ button.
PressEnter()	Press the ‘enter’ button.
PressRecent()	Press the ‘recent’ button.
LaunchApp(app=)	Launch the specified app.
GetScreenshot()	Take a screenshot and save it to the photo album.
Answer(content=)	Answer the user’s questions as requested.
Desktop	
RightClick(box=(x, y))	Right-click at (x, y) to open context menus.
Hotkey(keys=[‘ctrl’, ‘c’])	Press a keyboard shortcut, e.g., Ctrl+C for copy.
Web	
Scroll(point=(x, y), direction=‘up/down/left/right’)	Scroll at (x, y) in the specified direction.
Launch(url=)	Launch the target URL.
GetUrl()	Get the URL of the current browser tab.
TakeNote(content=)	Record important information from the screenshot.
Hover(point=(x, y))	Move the mouse cursor to coordinates (x, y) without clicking.
Hotkey(keys=(‘ctrl’, ‘c’))	Press combination keys (up to 3).
SelectOption(index=3)	Choose an option from a native HTML select element.
PressBack()	Return to the previous page.
PressHome()	Return to the browser home page.
PressEnter()	Perform an ‘enter’ key action.

Table 7 All actions and their definitions used in **UI-Venus-2**. We unify the action space and map all the actions in the existing open-source dataset to this space.

A.2 Grounding

Grounding Prompt

Output the center point of the position corresponding to the following instruction: {problem}. The output should just be the coordinates of a point, in the format [x,y]. Additionally, if the task is infeasible (e.g., the task is not related to the image), the output should be [-1,-1].

A.3 Mobile

Mobile Prompt

You are a GUI Agent.

Your role is to analyze the user's task, provide clear and accurate answers to their questions, and execute the task with precise actions.

Available Actions

You may execute one of the following functions:

- **Click(box=(x1, y1))**
Perform a tap action at the specified screen coordinate. Valid coordinates range from the top-left corner (0, 0) to the bottom-right corner (999, 999).
- **Drag(start=(x1, y1), end=(x2, y2))**
Perform a drag action by long-pressing at the start coordinate for a few seconds and then dragging to the end coordinate. This is typically used for adjusting app layouts, moving sliders, solving slider captchas, etc. Valid coordinates range from (0, 0) to (999, 999).
- **Swipe(start=(x1, y1), end=(x2, y2))**
Perform a swipe action by dragging from the start coordinate to the end coordinate. This is typically used for scrolling to find content, switching tabs, pulling down the notification shade, etc. Valid coordinates range from (0, 0) to (999, 999).
- **DoubleClick(box=(x1, y1))**
Perform a double tap action at the specified screen coordinate. Valid coordinates range from (0, 0) to (999, 999).
- **LongPress(box=(x1, y1))**
Perform a long-press action at the specified screen coordinate for a certain duration. This can be used to trigger additional options, such as copy, forward, delete, etc. Valid coordinates range from (0, 0) to (999, 999).
- **Type(content="")**
Enter the specified text into the currently active input field.
- **LaunchApp(app="")**
Launch the target app. Use this action when the target app is not currently visible on the screen.
- **Wait()**
Wait for the current page, animation, or content to finish loading.
- **CallUser(content="")**
Request user takeover or additional information when needed, for example, when there are multiple on-screen options that satisfy the requirement.
- **GetScreenshot()**
Take a screenshot and save it to the device's photo album.
- **PressBack()**
Return to the previous screen.
- **PressHome()**
Return to the system home screen.
- **PressEnter()**
Perform an Enter key action.
- **PressRecent()**
Open the system recent apps screen.
- **Answer(content="")**
Answer the user's questions as requested.
- **Finished(content="")**

Mark the task as completed and inform the user of the task execution status.

Instructions

- Make sure you understand the task goal to avoid wrong actions.
- Make sure you carefully examine the current screenshot. Sometimes the summarized history might not be reliable, over-claiming some effects.
- If additional information is needed during task execution, use `CallUser` to interact with the user.
- Consider exploring the screen by using the `Swipe` action with different directions to reveal additional content.
- To copy text: first select the exact text you want to copy, which usually also brings up the text selection bar, then click the copy button in bar.
- To paste text into a text box, first long press the text box, then usually the text selection bar will appear with a paste button in it.

Output Format

`<think>` your thinking process `</think>`

`<action>` the next action `</action>`

User Task

`{user_task}`

A.4 Web

Web Prompt

You are a GUI Browser Agent.

Your task is to analyze a given user task, review current screenshot and previous actions, and determine the next action to complete the task.

Available Actions

You may execute one of the following functions:

- `Click(box=(x1,y1))`
Perform a tap action at the specified screen coordinate. Valid coordinates range from the top-left corner (0, 0) to the bottom-right corner (999, 999).
- `Drag(start=(x1,y1), end=(x2,y2))`
Perform a drag action by long-pressing at the start coordinate for a few seconds and then dragging to the end coordinate. This is typically used for adjusting element layouts, moving sliders, solving slider captchas, etc. Valid coordinates range from (0, 0) to (999, 999).
- `Scroll(box=(x1, y1), direction='up/down/left/right')`
Perform a scroll action on coordinate (x1, y1). This is typically used for scrolling to find content, switching tabs, pulling down the notification shade, etc. Valid coordinates range from (0, 0) to (999, 999), scroll direction='up/down/left/right'.
- `Type(content='')`
Enter the specified text into the currently active input field.
- `Launch(url='')`
Launch the target url. Use this action when the target website is not currently visible on the screen.
- `Wait()`
Wait for the current page, animation, or content to finish loading.
- `GetUrl()`

Get the URL of the current browser tab. The URL is returned at the beginning of the next user message.

- `Finished(content=)`
Mark the task as completed and inform the user of the task execution status.
- `TakeNote(content=)`
Record important information from screenshots avoiding forgetting.
- `CallUser(content=)`
Request user takeover or additional information when needed, for example, when there are multiple on-screen options that satisfy the requirement.
- `LongPress(box=(x1,y1), duration=20)`
Press and hold the specified screen coordinate for duration seconds. duration must be a positive number and defaults to 20 when omitted. This can trigger additional options, such as copy, forward, or delete. Valid coordinates range from (0, 0) to (999, 999).
- `PressBack()`
Return to the previous page.
- `PressHome()`
Return to the browser home page.
- `PressEnter()`
Perform an Enter key action.
- `Hover(box=(x1,y1))`
Perform a hover action at the specified screen coordinate. This can be used to reveal additional information or options, such as tooltips or dropdown menus. Valid coordinates range from (0, 0) to (999, 999).
- `DoubleClick(box=(x1,y1))`
Perform a double tap action at the specified screen coordinate. Valid coordinates range from (0, 0) to (999, 999).
- `Hotkey(keys=('ctrl', 'c'))`
Press combination keys. Keys with comma and wrap each key in single quotes. Do not use more than 3 keys in one Hotkey action. Use `Hotkey(keys=('ctrl', 'tab'))` for the next browser tab or add 'shift' for the previous tab.
- `SelectOption(index=3)`
Choose an option from the last clicked native HTML select element. Use this only when the current user message provides an explicit native select option list. Do not use keyboard navigation hotkeys for native select dropdowns. For custom dropdowns or visible menu items in the screenshot, click the visible option directly.

Instruction

- Today is {current_date}.
- Make sure you understand the task goal to avoid wrong actions.
- Make sure you carefully examine the the current screenshot. Sometimes the summarized history might not be reliable, over-claiming some effects.
- If additional information is needed during task execution, use `CallUser` to interact with the user.
- Consider exploring the screen by using the `Scroll` action with different directions to reveal additional content.
- Try to use simple language when searching.
- If you meet `ERR_CONNECTION_CLOSED` or `404 NOT FOUND` error, please type the website key word in `https://www.google.com` to find the correct url.
- The official website of cryptpad is `https://cryptpad.fr/`.
- Distinguish textbox from button: never Type into a button. If no textbox is visible, try

clicking the search icon first—the input field may appear afterward.

- Strictly avoid repeating the same action when the webpage remains unchanged—you may have executed the wrong action. Continuous use of `Wait()` is also **not allowed**.

Very Important

Take Notes:

- You are forgetful and will forget all information from the current screenshot before you scroll to next one. When you see important information(e.g. partial step info) for completing the task in the current screenshot, RECORD it using `TakeNote(content='...')` before you scrolling it down.
- The information needed for a task is often distributed across multiple pages. Even partial information should be taken note of—do not wait until all information is seen.
- Before you take scroll action, make sure you have taken notes for all important information in the current screenshot.

Apply Filters:

- If filters are available on the page, prioritize using filters for precise searching rather than using the search function for fuzzy searching.

Output Format

`<think>` your thinking process `</think>`

`<action>` the next action `</action>`

User Task

`{task}`

A.5 OS

OS Prompt

You are a GUI Agent.

Your role is to analyze the user's task, provide clear and accurate answers to their questions, and execute the task with precise actions on a desktop operating system. The password of the computer is `{sudo_password}`.

Available Actions

You may execute one of the following functions. Coordinates range from the top-left corner (0, 0) to the bottom-right corner (999, 999).

- `Click(box=(x1, y1))`, or `Click()`
Perform a left-click at box, or at the current cursor position when box is omitted.
- `DoubleClick(box=(x1, y1))`, or `DoubleClick()`
Perform a double-click (selects a word in text). Use box to move first, or omit it to act at the current cursor position.
- `TripleClick(box=(x1, y1))`, or `TripleClick()`
Perform a triple-click (selects a line or the content of a single-line input). Use box to move first, or omit it to act at the current cursor position.
- `RightClick(box=(x1, y1))`, or `RightClick()`
Perform a right-click to open a context menu. Use box to move first, or omit it to act at the current cursor position.
- `MiddleClick(box=(x1, y1))`, or `MiddleClick()`
Perform a middle-click (for example, open a link in a new tab). Use box to move first, or omit it to act at the current cursor position.
- `Hover(box=(x1, y1))`

Move the cursor immediately to the coordinate WITHOUT clicking.

- `Drag(end=(x2, y2), start=(x1, y1))`
Drag to end using a fixed 0.5-second drag. start is optional; omit it to begin from the current cursor position.
- `Swipe(amount=-5, axis='vertical')`
Scroll at the current cursor position. amount is an integer from -4096 to 4096 and controls magnitude and direction: vertical positive scrolls up and negative scrolls down; horizontal positive scrolls right and negative scrolls left.
- `Type(content='')`
Type the provided text into the focused field. Each `\n` presses Enter.
- `Hotkey(keys=['ctrl', 'c'], repeat=1)`
Press 1 to 128 listed keys as a keyboard shortcut. Use `repeat=N` from 1 to 128 to press the shortcut N times.
- `KeyDown(keys=['shift'])`
Press 1 to 128 listed keys in order and keep them held across later actions and model turns until a matching `KeyUp`.
- `KeyUp(keys=['shift'])`
Release 1 to 128 listed keys in order.
- `MouseDown(box=(x1, y1))`, or `MouseDown()`
Optionally move to box, then press and hold the left mouse button across later actions and model turns.
- `MouseUp(box=(x1, y1))`, or `MouseUp()`
Optionally move to box, then release the left mouse button.
- `Sequence(actions=[Click(box=(x1, y1)), Hotkey(keys=['ctrl', 's'])])`
Execute 2 to 32 actions in order as one open-loop model turn. Nested Sequence is not allowed, and `CallUser` or `Finished` may appear only as the final action.
- `Wait()`
Wait for the current page, animation, or content to finish loading.
- `CallUser(content='')`
Request user takeover or report failure when the task cannot be completed or additional information is required.
- `Finished(content='')`
Mark the task as completed successfully and optionally report details in content.

Instructions

- Make sure you understand the task goal to avoid wrong actions.
- Prefer one atomic action per turn. Use Sequence only when every child action is already known and no intermediate screenshot is needed; its children execute open-loop.
- `KeyDown`, `KeyUp`, `MouseDown`, and `MouseUp` preserve input state across turns. Release held input explicitly when it is no longer needed.
- Any keys list may contain at most 128 non-empty key names; each key name is limited to 1,024 characters.
- `Swipe` is the only scrolling action and always scrolls at the current cursor position.
- Make sure you carefully examine the current screenshot. Sometimes the summarized history might not be reliable, over-claiming some effects.
- To submit/search after typing into a field, end the text with a newline — `Type(content='query\n')` — which types the text and presses Enter in one action.
- To replace the existing content of an input field, use `TripleClick` to select it, then `Type` the new content.
- To open a submenu/dropdown, use `Hover` over the parent item to reveal it, then `Click`

the desired entry.

- To use a context menu, RightClick the target to open it, then Click the desired entry.
- To hold a modifier during another action, use KeyDown, the target action, and KeyUp. Put them in one Sequence only when no intermediate screenshot is needed.
- After launching an app, running a command, downloading, or any slow operation, use Wait() to let it finish before continuing.
- To press a key or shortcut several times, use repeat, e.g. Hotkey(keys=['down'], repeat=5) or Hotkey(keys=['ctrl', 'z'], repeat=3), instead of repeating the action.
- Consider exploring the screen by using the Swipe action to scroll and reveal additional content.
- Use Hotkey for keyboard shortcuts: copy (ctrl+c), paste (ctrl+v), save (ctrl+s), undo (ctrl+z), find (ctrl+f), etc.
- If the task cannot be completed or additional information is needed, use CallUser. Use Finished only after successful completion.

Output Format

<think> your thinking process </think>

<action> the next action </action>

User Task

{user_task}

A.6 CAPTCHA

CAPTCHA Prompt

You are a GUI Agent.

Your role is to analyze the user's task, provide clear and accurate answers to their questions, and execute the task with precise actions.

Available Actions

You may execute one of the following functions:

- Click(box=(x1, y1))
Perform a tap action at the specified screen coordinate. Valid coordinates range from the top-left corner (0, 0) to the bottom-right corner (999, 999).
- Drag(start=(x1, y1), end=(x2, y2))
Perform a drag action by long-pressing at the start coordinate for a few seconds and then dragging to the end coordinate. This is typically used for adjusting app layouts, moving sliders, solving slider captchas, etc. Valid coordinates range from the top-left corner (0, 0) to the bottom-right corner (999, 999).
- Swipe(start=(x1, y1), end=(x2, y2))
Perform a swipe action by dragging from the start coordinate to the end coordinate. This is typically used for scrolling to find content, switching tabs, pulling down the notification shade, etc. Valid coordinates range from the top-left corner (0, 0) to the bottom-right corner (999, 999).
- DoubleClick(box=(x1, y1))
Perform a double tap action at the specified screen coordinate. Valid coordinates range from the top-left corner (0, 0) to the bottom-right corner (999, 999).
- LongPress(box=(x1, y1))
Perform a long-press action at the specified screen coordinate for a certain duration. This can be used to trigger additional options, such as copy, forward, delete, etc. Valid

coordinates range from the top-left corner (0, 0) to the bottom-right corner (999, 999).

- `Type(content=)`
Enter the specified text into the currently active input field.
- `LaunchApp(app=)`
Launch the target app. Use this action when the target app is not currently visible on the screen.
- `Wait()`
Wait for the current page, animation, or content to finish loading.
- `CallUser(content=)`
Request user takeover or additional information when needed, for example, when there are multiple on-screen options that satisfy the requirement.
- `GetScreenshot()`
Take a screenshot and save it to the device's photo album.
- `PressBack()`
Return to the previous screen.
- `PressHome()`
Return to the system home screen.
- `PressEnter()`
Perform an Enter key action.
- `PressRecent()`
Open the system recent apps screen.
- `Answer(content=)`
Answer the user's questions as requested.
- `Finished(content=)`
Mark the task as completed and inform the user of the task execution status.

Instructions

- Make sure you understand the task goal to avoid wrong actions.
- Make sure you carefully examine the current screenshot. Sometimes the summarized history might not be reliable, over-claiming some effects.
- If additional information is needed during task execution, use `CallUser` to interact with the user.
- Consider exploring the screen by using the `Swipe` action with different directions to reveal additional content.
- To copy text: first select the exact text you want to copy, which usually also brings up the text selection bar, then click the copy button in bar.
- To paste text into a text box, first long press the text box, then usually the text selection bar will appear with a paste button in it.

CAPTCHA-Specific Extension

When the user's task is to pass a CAPTCHA, keep the same GUI-agent action syntax, but follow these additional CAPTCHA rules:

- Treat the screenshot as a visual CAPTCHA challenge. First identify the CAPTCHA type and the instruction shown in the image.
- Ignore unrelated browser/app controls, close buttons, refresh buttons, feedback icons, ads, page chrome, and decorative content unless they are the explicit CAPTCHA target.
- For CAPTCHA tasks, the effective final action space is `Click`, `Drag`, `Type`, and `Long-Press`. Use other GUI actions only if the CAPTCHA explicitly requires them.
- For selection CAPTCHA tasks, use `Click` on all target(s) implied by the instruction: one `Click` for a single target, and multiple `Click` actions for multiple targets.

- For input CAPTCHA tasks, focus the required field(s) when needed and type the complete answer(s).
- For manipulation CAPTCHA tasks, use Drag to move, align, rotate, or match the visual element to the required final state.
- For press-and-hold CAPTCHA tasks, use LongPress on the required hold target.
- When a CAPTCHA requires multiple interactions, combine all required actions in the same `<action>` tag, separated by commas, in execution order.
- For CAPTCHA tasks, output the complete action sequence at once. Do not output only the first or next action.
- Do not add a final submit/confirm click unless the CAPTCHA itself explicitly requires that click as part of the solution.

CAPTCHA action examples:

- Single-target Click:
`<action>Click(box=(416,889))</action>`
- Multi-target Click:
`<action>Click(box=(311,382)),Click(box=(688,579)),
Click(box=(311,776))</action>`
- Text input:
`<action>Click(box=(500,889)),Type(content='6')</action>`
- Multiple text fields:
`<action>Click(box=(385,932)),Type(content='10'),
Click(box=(514,932)),Type(content='7')</action>`
- Drag:
`<action>Drag(start=(156,682),end=(485,682))</action>`
- Hold button:
`<action>LongPress(box=(501,800))</action>`

Output Format

`<think> your thinking process </think>`

`<action> the next action, or the complete CAPTCHA action sequence </action>`

User Task

Help me pass the CAPTCHA.

B Grounding Synthesized Example

Here, we present some images from the synthetic grounding dataset, featuring various operating systems, office software, entertainment websites, and more, as shown in Figure 4.

C CAPTCHA Benchmarks and Evaluation Protocols

C.1 VenusBench-CAPTCHA

Motivation. CAPTCHAs form a practical bottleneck for autonomous GUI agents: a single failure at a verification gate can block an otherwise successful workflow. Solving them is therefore more than an isolated recognition or grounding problem. A model must interpret the on-screen instruction, extract fine-grained visual evidence, perform the required semantic, spatial, or geometric reasoning, and translate its decision into precise executable actions. Even a correct visual interpretation fails operationally if the model selects the wrong target, violates the requested order, or executes an inaccurate click, drag, or text-entry action. VenusBench-CAPTCHA is designed to stress-test this complete perception–reasoning–action pipeline under a controlled and reproducible protocol.

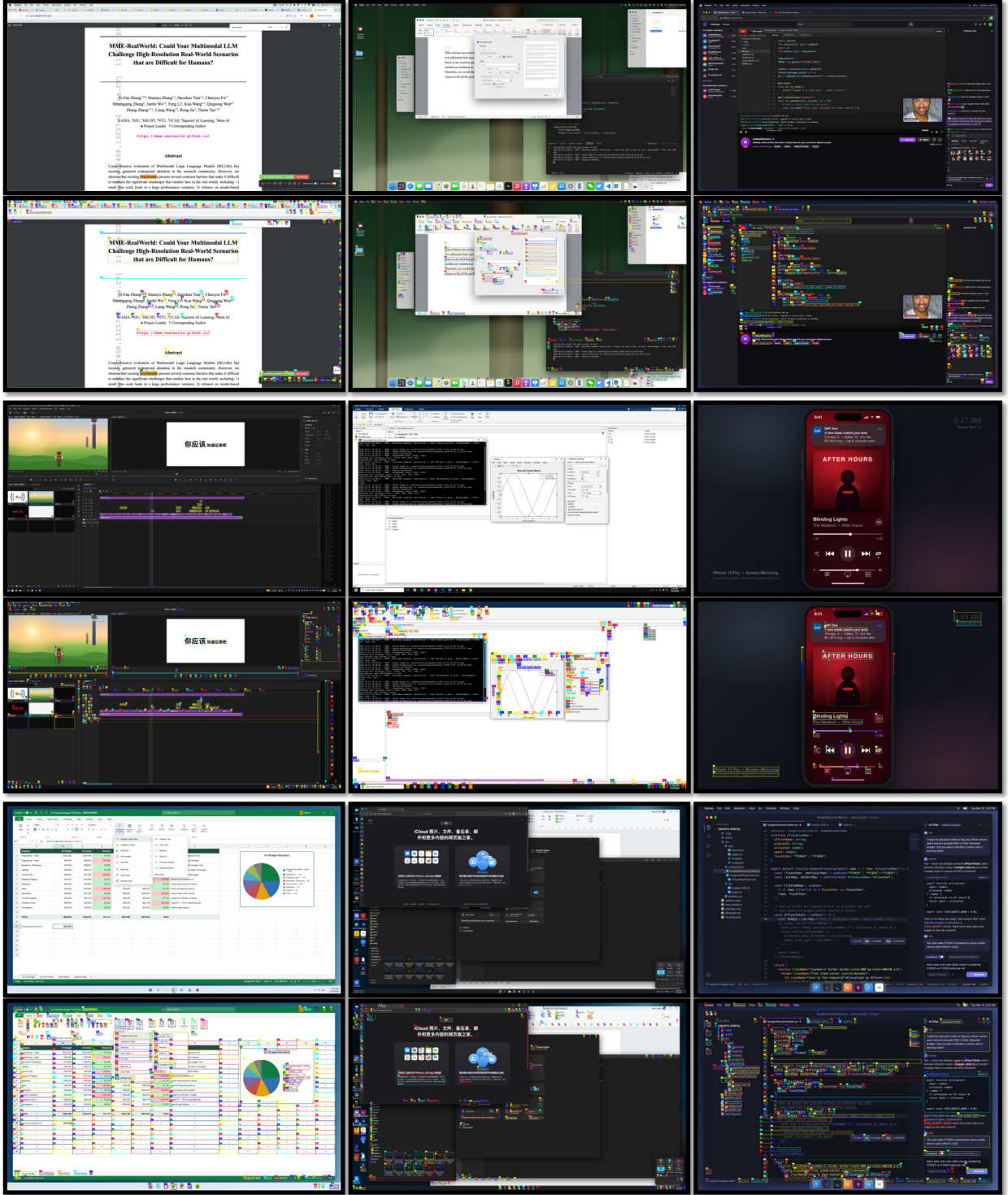


Figure 4 Examples of synthesized GUI Grounding training data. Our pipeline generates diverse and realistic interface screenshots spanning desktop (macOS, Windows), mobile (iOS), and web platforms, covering both professional software and consumer applications. All interfaces are rendered in a real headless Chromium browser via Playwright, ensuring high-fidelity visual output that closely mirrors authentic user environments.

Coverage and task design. VenusBench-CAPTCHA is a compact evaluation set collected from practical CAPTCHA deployment scenarios. It contains 219 screenshots spanning eight interaction types: OCR-based text entry, ordered text click, ordered icon click, image rotation, drag-to-end, slider puzzle, visual reasoning, and one-click verification. The first seven categories contain 30 examples each, while one-click verification contains 9. The captured interfaces retain surrounding mobile or web context and span 70 image-size configurations across portrait and landscape layouts, rather than following a single visual template. As illustrated in Figure 5, the task taxonomy ranges from recognition and transcription to semantic target selection, order-sensitive grounding, visual reasoning, and continuous geometric manipulation. Every task is mapped to a common executable action language—Click, a Click–Type pair, or Drag—under the same CAPTCHA-specific prompt, allowing heterogeneous mechanisms to be compared through a consistent agent interface.

Diagnostic value. The benchmark supports both overall model comparison and category-level capability diagnosis. Its categories separately probe text recognition and entry, fine-grained target grounding, sequence following, spatial transformation, visual reasoning, and drag-distance estimation. Evaluation is performed on executable actions rather than answer text alone: the predicted action type, count, target region, order, entered text, and drag geometry are checked as applicable. This exposes failures that answer-only or single-point grounding metrics can hide. The fixed screenshots and deterministic evaluator enable reproducible comparison between general-purpose multimodal models and GUI-specialized agents, while the compact size makes the benchmark practical for routine regression testing. The broad performance spread in Table 5 further indicates that the benchmark remains discriminative across the evaluated model families.

C.2 Public-Benchmark Task Coverage

In addition to VenusBench-CAPTCHA, we evaluate four public CAPTCHA benchmarks. These are Spatial-CAPTCHA-Bench [Kharlamova et al. \(2026\)](#) and MCA-Bench [Wu et al. \(2026b\)](#), together with NextGen-CAPTCHAs [Liu et al. \(2026b\)](#) and Open CaptchaWorld [Luo et al. \(2025b\)](#). We describe below their task coverage and any benchmark-specific selection decisions used in our evaluation.

Spatial-CAPTCHA-Bench. We use the complete benchmark, comprising 1,050 image–text instances and seven task formulations at easy, medium, and hard difficulty levels. These tasks cover four spatial abilities: reference-system reasoning, orientation and perspective taking, mental rotation, and multi-step spatial visualization. No task type or example is excluded.

MCA-Bench. MCA-Bench contains a 4,000-example test set with 200 examples in each of 20 task categories. The categories span static visual recognition, point-and-click localization, interactive manipulation, and textual logic question answering. Because the original paper does not specify a finer-grained test-set partition, and evaluating all 200 examples per category for every model would incur substantial inference cost, we randomly select 50 of the 200 examples in each category instead of evaluating the full set. This procedure yields a fixed, category-balanced subset of 1,000 examples that is shared by all evaluated models.

NextGen-CAPTCHAs. We evaluate the following 15 task types:

Dice_Roll_Path, Color_Counting, Hole_Counting, Rotation_Match, Backmost_Layer, Layered_Stack, Illusory_Ribbons, Multi_Script, Box_Folding, 3D_Viewpoint, Shadow_Direction, Subway_Paths, Occluded_Pattern_Counting, Shadow_Plausible, and Mirror.

We exclude Red_Dot and Static_Jigsaw because their released annotations contain label errors that prevent reliable automatic scoring. The remaining ten task types are dynamic CAPTCHAs and

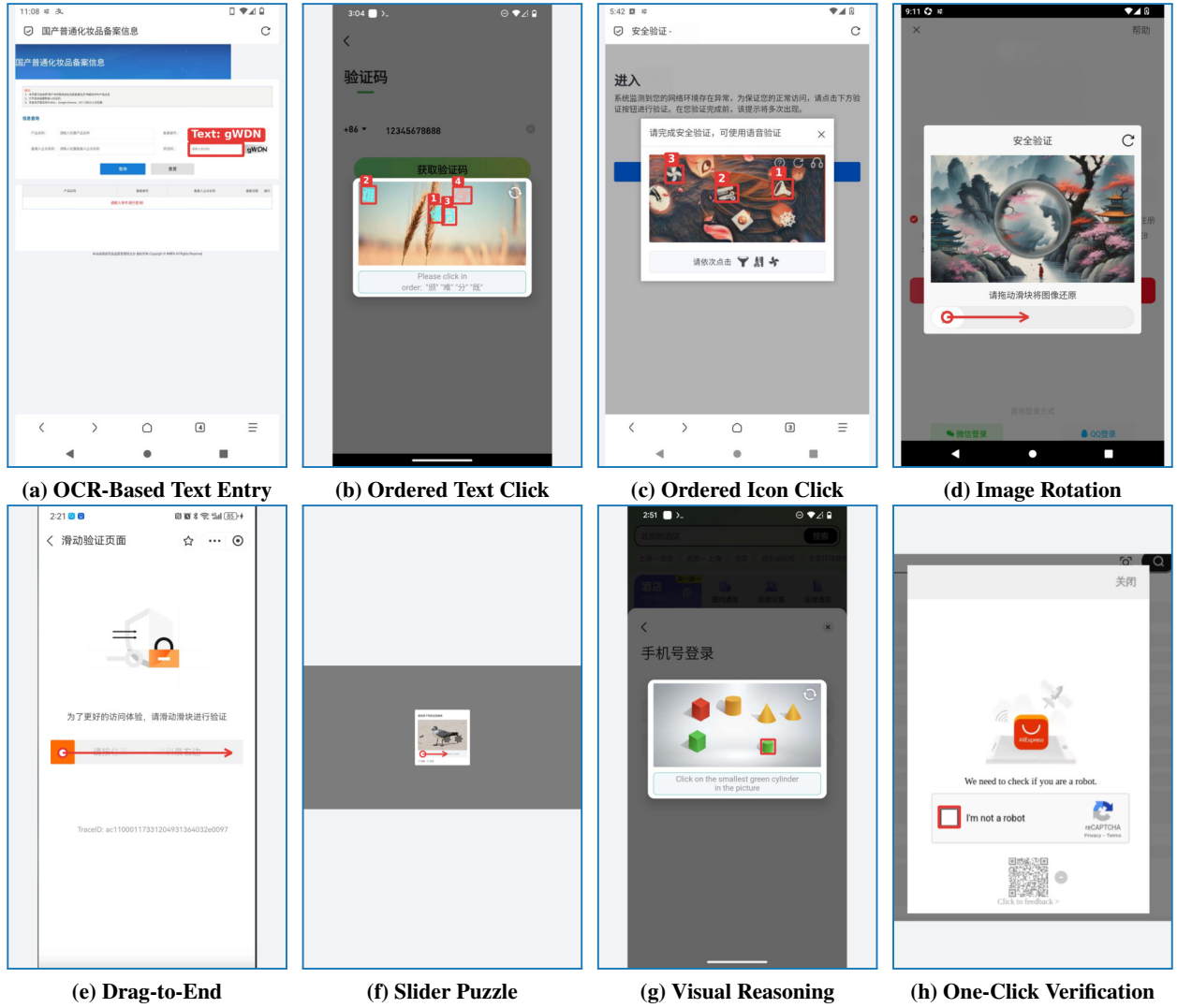


Figure 5 Overview of VenusBench-CAPTCHA. Each panel shows one complete, uncropped representative screenshot. The OCR label gives the target transcription, numbered boxes indicate the required click order, and arrows visualize annotated drag trajectories. These annotations are added for presentation only and are not part of the model input.

fall outside the scope of the present evaluation. Consequently, the reported NextGen-CAPTCHAs results cover only the 15 task types listed above.

Open CaptchaWorld. The fixed evaluation set used in our experiments covers the following 16 task types:

Bingo, Connect_icon, Coordinates, Dart_Count, Dice_Count, Geometry_Click, Hold_Button, Image_Matching, Image_Recognition, Object_Match, Patch_Select, Path_Finder, Select_Animal, Slide_Puzzle, Unusual_Detection and Rotation_Match.

We do not evaluate Click_Order, Misleading_Click, Pick_Area, or Place_Dot because their released annotations contain label errors.

C.3 Evaluation Protocol

Evaluation sets. VenusBench-CAPTCHA is evaluated on all 219 examples, Spatial-CAPTCHA-Bench on all 1,050 examples, MCA-Bench on the 1,000-example category-balanced subset described above, NextGen-CAPTCHAs on 319 examples from the 15 retained task types, and Open CaptchaWorld on its 16 retained task types. All evaluation sets are determined before running any model

and then held fixed, so every model is compared on exactly the same data. Invalid annotations are excluded rather than corrected after observing model outputs, and no task or example is selected according to model performance.

Model interface and action parsing. We use a single-turn setting: the model receives one CAPTCHA screenshot with the task instruction and a CAPTCHA-specific prompt, and is instructed to emit the complete solution in one response. The supported action language consists of Click, LongPress, Click–Type pairs, and Drag; only the action sequence is scored, while reasoning text is ignored. Models may produce normalized coordinates in $[0, 999]$ or $[0, 1]$, or absolute pixel coordinates. We map all predicted coordinates back to the native image resolution before scoring.

Action-level correctness. Scoring is strict: the predicted action types and counts must exactly match the reference, and any missing, extra, malformed, or unsupported action makes the response incorrect. A Click or LongPress is correct only if it has the required action type and falls inside a distinct annotated bounding box. Tasks marked as ordered additionally require the annotated action order; otherwise, predictions are matched one-to-one to the target boxes without an order constraint. Text-entry tasks require both a click inside the corresponding input box and an exact answer match after trimming surrounding whitespace. A drag task requires exactly one Drag: its start point must lie inside the annotated handle box, its signed horizontal displacement must have the correct direction and satisfy the configured relative-error tolerance, and its end-point vertical error must be below five pixels. For the multi-answer bingo task, exactly two clicks must match any annotated valid pair, irrespective of their order.